Master's Degree Program

Field of Study: Advanced Analysis – Big Data

Thi Van Trang Le
Number ID : 140036

Process Intelligence with Python Language

Master's Thesis
Under the supervision of
dr hab. Michała Ramsza
Institute of Mathematical Economics

Warszawa 2026

Contents

Chapter 1

Introduction

1.1 Background and motivation

Modern organizations carry out much of their work through digital platforms such as Enterprise Resource Planning (ERP) systems, Customer Relationship Management (CRM) tools, and dedicated workflow management applications. As these systems operate, they automatically generate a continuous stream of structured records that capture every action performed within a business process. Each record typically stores information about which activity was performed, who performed it, when it occurred, and which business object it concerned. Over time, these records accumulate into large repositories of operational data that reflect the actual behavior of business processes in precise detail.

This type of data offers a qualitatively different view of business operations compared with traditional management reports. A management report typically presents aggregated statistics — average throughput times, error rates, or resource utilization figures — computed from a simplified model of how the process should work. By contrast, the raw event data generated by a digital information system describes how the process actually works, case by case and activity by activity. The gap between these two views is often significant. Studies have consistently shown that real business processes deviate from their intended designs in ways that are difficult to detect from aggregate data alone (Aalst, 2022). Bottlenecks, rework cycles, policy violations, and unofficial workarounds are all examples of behavior that is invisible in summary reports but clearly visible in event data when it is analyzed correctly.

Process mining is the scientific discipline that bridges this gap. It applies computational methods to event logs — collections of recorded process events — in order to discover how a process is structured, verify whether it follows established rules, and measure its operational performance. Since its formal introduction in the late 1990s, process mining has grown from a niche research topic into a well-established field with a substantial body of theory, a dedicated international conference, and a growing number of commercial and open-source software tools (Aalst, 2022). Today it is applied in industries including healthcare, finance, logistics, and manufacturing, where it supports tasks such as compliance auditing, operational improvement, resource management, and process redesign.

Process intelligence is a broader term that encompasses process mining but extends it in several important directions. While process mining is primarily retrospective — it analyzes data that has already been recorded — process intelligence also includes real-time monitoring of running processes, predictive modeling to anticipate future outcomes, simulation of alternative process designs, and optimization of process parameters. This combination of retrospective analysis, real-time awareness, and forward-looking

capabilities represents the current state of the art in data-driven process management. Organizations that adopt process intelligence are not limited to understanding what went wrong in the past; they can also intervene in ongoing processes and proactively improve future performance.

Python has become the dominant language for data science and analytical computing over the past decade, and this has made it a natural platform for implementing process mining and process intelligence workflows. The Python ecosystem includes not only general-purpose tools for data manipulation, statistical analysis, and machine learning, but also a growing set of libraries specifically designed for process mining. The most prominent of these is PM4Py, an open-source library that implements the full process mining workflow — from loading and exploring event logs to discovering process models, checking conformance, and analyzing performance. Other libraries extend these capabilities to include simulation, prediction, and interactive visualization. Together they form an ecosystem that makes it possible to conduct end-to-end process intelligence analysis within a single Python environment, without reliance on proprietary platforms or specialized software.

Despite this growth, the Python process mining ecosystem remains fragmented and difficult to navigate for a practitioner who is encountering it for the first time. Libraries have overlapping functionalities, varying levels of documentation, and different assumptions about data formats and analytical workflows. No systematic and practical comparison of these tools currently exists that covers the breadth of available libraries and illustrates their use with concrete, reproducible examples. There is therefore a clear need for a structured review that organizes these tools, explains their theoretical foundations, and demonstrates their capabilities with working implementations.

1.2 Research objectives and research question

The central research question of this thesis is:

Which Python tools are available for process mining and process intelligence, and how suitable are they for key analytical tasks such as process discovery, conformance checking, and performance analysis?

To address this question, the thesis pursues four specific objectives:

1. To introduce and formalize the mathematical concepts that underlie process mining — including event logs, traces, directly-follows relations, process models, and conformance measures — so that the capabilities and limitations of different tools can be understood and compared on a rigorous basis.
2. To survey the main Python libraries available for process mining and process intelligence, organized by their primary function, and to describe their features, strengths, and limitations in a structured way.
3. To demonstrate the use of selected tools through practical worked examples, including a synthetic introductory example and a real-world case study based on a publicly available benchmark dataset.

4. To draw conclusions about the current state of the Python process mining ecosystem and to identify gaps and directions for future development.

The scope of the thesis is limited to Python tools and to the three core process mining tasks — discovery, conformance checking, and performance analysis — along with adjacent capabilities such as simulation and predictive monitoring. The focus is on open-source tools that are accessible to practitioners and researchers without requiring commercial licenses, with the exception of PyCelonis, which is included because it represents an important category of commercial process intelligence platform with a Python interface.

1.3 Relevance and academic context

Process mining as a scientific field was formally established around the work of Wil van der Aalst and colleagues at Eindhoven University of Technology in the late 1990s and early 2000s. The field has since grown into a mature discipline with dedicated academic infrastructure: the International Conference on Process Mining (ICPM) is held annually, and the IEEE Task Force on Process Mining has produced a formal manifesto defining the scope and principles of the field. The textbook by van der Aalst (Aalst, 2022) provides a comprehensive reference for both the theoretical foundations and the practical applications of process mining and is the primary academic reference for this thesis.

The BPI Challenge, organized annually in connection with ICPM, provides publicly available benchmark datasets from real organizations and invites researchers to analyze them using process mining methods. These datasets have become standard benchmarks in the process mining literature and are widely used to compare algorithms, tools, and analytical approaches. The BPI Challenge 2019 dataset, which is used as the main case study in this thesis, was collected from a real procurement process at a large multinational company. It contains nearly 1.6 million events and represents one of the most complex datasets in the BPI Challenge series. It has been analyzed in numerous academic papers, which makes it a suitable choice for a thesis that aims to demonstrate process mining techniques in a realistic setting.

The Python ecosystem for process mining has developed rapidly since the initial release of PM4Py in 2018. Earlier process mining software was implemented primarily in ProM, a Java-based platform that remains widely used in academic research but requires specialized knowledge to extend and deploy. The availability of a Python implementation of the core algorithms significantly lowered the barrier to entry and enabled process mining to be integrated with the broader Python data science ecosystem, including libraries for machine learning, visualization, and statistical analysis. Since then, additional libraries have been released to support specific tasks such as streaming process mining, simulation-based optimization, and predictive monitoring. However, the ecosystem remains less mature and less comprehensively documented than the equivalent Java-based ProM ecosystem, which makes a structured review particularly timely.

This thesis contributes to this context by providing a structured, practitioner-oriented review of the Python process mining ecosystem, with particular attention to PM4Py and its extensions. The practical examples are implemented as Jupyter notebooks, which allows all steps to be reproduced and verified by the reader. This reproducibility is an important feature of the work, as it ensures that the results reported in Chapter 4 can be independently validated and extended by other researchers or practitioners.

1.4 Methodology

This thesis is a structured review combined with empirical demonstration. The work follows four sequential phases.

Phase 1: Theoretical foundation. The mathematical concepts required to understand process mining are introduced and formalized in Chapter 2. The primary source for these definitions is van der Aalst (Aalst, 2022). Establishing this theoretical foundation before describing tools is important because it provides a common vocabulary for comparing different implementations and for interpreting the results of the practical examples.

Phase 2: Tool survey. Python libraries are identified through a combination of academic sources — papers and conference proceedings from ICPM and related venues — the Python Package Index (PyPI), GitHub repositories related to process mining and process intelligence, and cross-references within the PM4Py documentation. For each tool, the following aspects are examined: the primary analytical task it supports, the data formats it accepts, the algorithms it implements, and the level of active maintenance and community support as indicated by recent commit history and documentation quality. Tools are organized into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages.

Phase 3: Practical implementation. Two examples are implemented in Python using PM4Py as the primary tool, with supporting libraries as needed. The first is a synthetic introductory example that constructs a small event log by hand and illustrates the basic process mining workflow on data where the correct answer is known in advance. The second is a case study using the BPI Challenge 2019 dataset, which applies the same workflow to a large, complex, real-world event log and reports the resulting process model and performance statistics. Both examples are implemented as Jupyter notebooks, which are embedded directly in the thesis so that the code, outputs, and commentary are presented together in a transparent and reproducible format.

Phase 4: Synthesis and conclusions. The results of the tool survey and the practical examples are combined in Chapter 5, where the research question is answered, the limitations of the work are acknowledged, and directions for future research are proposed.

The methodology is descriptive and demonstrative rather than evaluative in a strict experimental sense. No formal benchmarking is performed — the thesis does not measure and compare the runtime, fitness scores, or precision values of multiple discovery

algorithms on the same dataset. Such a comparison would require a controlled experimental design and falls outside the scope of this work. The contribution instead lies in the structured organization, explanation, and demonstration of the available tools.

1.5 Structure of the thesis

The remainder of this thesis is organized as follows.

Chapter 2 Introduces the mathematical foundations of process mining. It formally defines events, traces, and event logs, explains directly-follows relations and their role in process discovery, describes the main process model representations used in this thesis — the directly-follows graph, the process tree, and the Petri net — and introduces the concepts of conformance checking and performance analysis. It also explains the structure of the Object-Centric Event Log (OCEL) format and provides an exploratory description of the BPI Challenge 2019 dataset.

Chapter 3 Reviews the main Python tools available for process mining and process intelligence. The tools are organized into five categories — core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages — and each category is described in terms of the tasks it supports and the tools it contains. The chapter concludes with a comparative summary table.

Chapter 4 Presents three worked examples. The first is a synthetic introductory example that demonstrates the basic process mining workflow on a small hand-constructed dataset. The second is a full case study based on the BPI Challenge 2019 dataset, covering data loading, exploratory analysis, flattening, process discovery, and result interpretation. The third extends the case study to include variant analysis, loop detection, and throughput time distribution analysis.

Chapter 5 Summarizes the main findings of the thesis, discusses its limitations, and proposes directions for future research.

Chapter 2

The mathematics of process mining

Process mining is built on a precise mathematical vocabulary. Without formal definitions, it is impossible to specify what a process model represents, what it means for a log to fit a model, or how two algorithms can be compared objectively. This chapter introduces the core mathematical concepts used throughout the thesis: events, traces, and event logs; directly-follows relations and variants; process discovery and process models; conformance checking; and performance analysis. Each concept is illustrated with a small worked example so that the formal definition connects directly to something concrete. The chapter also explains how event data are represented in Python and provides an exploratory description of the BPI Challenge 2019 dataset that is used in the case study in Chapter 4.

2.1 Event logs and traces

An event is the basic unit of data in process mining. Each event records that a specific activity was performed for a specific process instance at a specific point in time. Formally, let $\mathcal{U}_C$ be the universe of case identifiers, $\mathcal{U}_A$ be the universe of activity names, and $\mathcal{U}_T$ be the universe of timestamps, where $\mathcal{U}_T$ is totally ordered by the natural chronological ordering $\leq$.

An **event** e is a tuple $e = (c, a, t)$ where $c \in \mathcal{U}_C$ is a case identifier, $a \in \mathcal{U}_A$ is an activity name, and $t \in \mathcal{U}_T$ is a timestamp. In practice, events may carry additional attributes such as the name of the resource that performed the activity, the organizational unit, the cost, or business-specific data fields. These attributes extend the basic definition without changing its core structure. For the purposes of process discovery, only the case identifier, activity name, and timestamp are strictly required.

A **trace** is the projection of all events belonging to a single case onto the sequence of their activity names, ordered by timestamp. Let A be a finite set of activity names. A trace σ is a finite sequence of activity names written as $\sigma = \langle a_1, a_2, ..., a_n \rangle$, where each $a_i \in A$ and the order of elements follows the chronological order of the underlying events. The set of all finite sequences over A is denoted A^*.

An **event log** L collects traces from many cases. Because the same sequence of activities can occur in more than one case, the log is modelled as a multiset over A^*, written $L \in \mathcal{B}(A^*)$, where $\mathcal{B}$ denotes the multiset operator. A multiset is a function $m : X \to \mathbb{N}_0$ that assigns a non-negative integer multiplicity to each element of X. For a trace $\sigma \in A^*$, the value $L(\sigma)$ gives the number of cases in which that exact sequence was observed.

To make this concrete, consider the following nine events:

Event ID	Activity	Case ID	Timestamp
e1	Create order	1	09:00

Event ID	Activity	Case ID	Timestamp
e2	Approve order	1	10:00
e3	Send invoice	1	11:00
e4	Create order	2	09:15
e5	Approve order	2	10:30
e6	Send invoice	2	11:20
e7	Create order	3	08:45
e8	Reject order	3	09:50
e9	Close case	3	10:40

Grouping by case identifier and ordering by timestamp gives three traces:

- Case 1: $\sigma_1 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 2: $\sigma_2 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 3: $\sigma_3 = \langle \text{Create order}, \text{Reject order}, \text{Close case} \rangle$

Because $\sigma_1 = \sigma_2$, the event log is the multiset $L = [\sigma_1^2, \sigma_3^1]$. This log has two distinct trace patterns: the approval path (observed twice) and the rejection path (observed once). The total number of cases is $\|L\| = 3$, the number of distinct variants is 2, and the average trace length is $\frac{3+3+3}{3} = 3$.

Several summary statistics are routinely computed from a log before applying any discovery algorithm. The **number of distinct variants** is $|\{\text{Var}\}(L)| = |\{\sigma \in A^* : L(\sigma) > 0\}|$. The **average trace length** is $\bar{\ell} = \frac{\sum_{\sigma \in L} L(\sigma) \cdot |\sigma|}{\|L\|}$. The **variant frequency distribution** ranks traces by their multiplicity in descending order. In real-world logs this distribution is typically highly skewed: a small number of variants account for the large majority of cases, while many variants are observed only once or twice. This skewness is important because it guides decisions about filtering and sampling before applying discovery algorithms.

Traces are useful for studying both the structure and the variability of a process. When most cases follow the same sequence, the process is stable and predictable. When many distinct sequences appear, the process is more variable and may require a more flexible model to represent it accurately. The distribution of trace frequencies is therefore one of the first things an analyst examines when working with a new event log, as it gives an immediate indication of how complex and varied the underlying process is.

2.2 Activities, variants, and directly-follows relations

One of the most important structural relationships between activities in a log is the directly-follows relation. Activity b directly follows activity a in trace σ if a appears immediately before b in σ. Formally, for a trace $\sigma = \langle a_1, ..., a_n \rangle$, the relation $a \underset{\sigma}{>} b$ holds if there exists some index i such that $a_i = a$ and $a_{i+1} = b$. The directly-follows relation of the full log L is then:

$$\underset{L}{>} = \bigcup_{\sigma \in \text{ support}(L)} \underset{\sigma}{>} \tag{1}$$

The **frequency** of a directly-follows pair (a, b) counts how many times b immediately follows a across all traces in the log:

$$f_{L(a,b)} = \sum_{\sigma \in L} L(\sigma) \cdot |\{i : a_i = a, a_{i+1} = b\}| \tag{2}$$

The **directly-follows graph** (DFG) of a log L is the directed weighted graph $\mathrm{DFG}(L) = (A_L, E_L, f_L)$ where A_L is the set of activities appearing in at least one trace, $E_L = \underset{L}{>}$ is the set of directed edges, and f_L assigns each edge its frequency. In addition, a start node and an end node are included, with edges from the start node to every activity that begins at least one trace, and edges from every activity that ends at least one trace to the end node. These edges are weighted by the frequency of each start and end activity.

Using the log from the previous section, the directly-follows pairs and their frequencies are:

From activity	To activity	Frequency
Create order	Approve order	2
Approve order	Send invoice	2
Create order	Reject order	1
Reject order	Close case	1

The DFG reveals two distinct execution paths through the process: the approval path (Create → Approve → Send, frequency 2) and the rejection path (Create → Reject → Close, frequency 1). This already suggests the presence of an exclusive choice after Create order, which a process discovery algorithm would later formalize. The DFG provides a quick visual overview of which activities tend to follow each other and how often. However, it does not distinguish between control-flow patterns such as sequences, choices, and parallel branches, which limits its usefulness for deeper structural analysis.

A **variant** is any trace σ with $L(\sigma) > 0$. The variant set is $\mathrm{Var}(L) = \{\sigma \in A^* : L(\sigma) > 0\}$. Variant analysis ranks variants by frequency and examines the proportion of cases accounted for by the top-k variants. The **coverage** of the top k variants is:

$$\mathrm{Coverage}(k) = \frac{\sum_{i=1}^{k} L(\sigma_i^*)}{\|L\|} \tag{3}$$

where $\sigma_1^*, \sigma_2^*, \ldots$ are the variants ordered by decreasing frequency. Filtering the log to retain only the top-k variants is a standard preprocessing step that focuses the analysis on dominant behavior and reduces the influence of noise and exceptional cases. In the BPI Challenge 2019 dataset, the full log contains 26,378 distinct variants after flattening, illustrating how variable real procurement processes can be in practice.

Simple counts — the number of events per activity, the total number of cases, the number of distinct variants, and the average trace length — also provide a useful initial summary of the log. These statistics help the analyst understand the overall complexity and variability of the process before applying more detailed techniques.

2.3 Process discovery and process models

Process discovery is the task of constructing a process model directly from an event log, without using any prior reference model. A good discovered model must balance four quality dimensions:

- **Fitness** measures whether the model can reproduce the traces observed in the log. A model with high fitness accepts most of the behavior that actually occurred. A model with perfect fitness would accept every trace in the log. Formally, if $\mathcal{L}(M)$ denotes the language (set of accepted traces) of model M, then high fitness means $\text{Var}(L) \subseteq \mathcal{L}(M)$ or at least most of $\text{Var}(L)$ is covered.
- **Precision** measures whether the model avoids accepting behavior that was never observed. A model with low precision is too general and allows many traces that do not appear in the log. A model that simply allows any sequence of any activity has perfect fitness but zero precision.
- **Generalization** reflects how well the model handles cases that were not seen during discovery but are still plausible given the process structure. A model that memorizes every observed trace exactly without allowing for any unseen but valid behavior is said to overfit the log.
- **Simplicity** refers to how easy the model is to read and interpret. Simpler models are generally preferred when they still achieve acceptable fitness and precision. The most common simplicity measure is the total number of nodes and arcs in the model representation.

These four dimensions are in tension with each other. Improving fitness often reduces precision, since a more permissive model accepts a wider range of behavior. Finding an appropriate balance is one of the central challenges in process discovery, and different algorithms make different trade-offs among these dimensions. For large and noisy real-world logs, robustness to noise and computational efficiency are also practical considerations that influence algorithm selection.

2.3.1 The Alpha algorithm

The Alpha algorithm is one of the earliest and most well-known process discovery algorithms. It works by analyzing the directly-follows relations in the log and inferring causal dependencies between activities. From these dependencies, it constructs a Petri net that represents the discovered process.

The algorithm begins by computing four binary relations between all pairs of activities in A_L based on directly-follows information:

- $a \to b$ (direct succession): b directly follows a in at least one trace
- $a \gg b$ (causality): $a \to b$ holds but $b \to a$ does not — a appears to cause b
- $a \parallel b$ (parallelism): both $a \to b$ and $b \to a$ hold — the two activities can appear in either order
- $a \# b$ (independence): neither $a \to b$ nor $b \to a$ holds — the activities never directly follow each other

The algorithm then identifies all pairs (A, B) of non-empty activity sets where every activity in A causes every activity in B (i.e., $a \gg b$ for all $a \in A, b \in B$), all activities within A are independent of each other, and all activities within B are independent of each other. The maximal such pairs become places in the discovered Petri net, each connecting the activities in A as input transitions to the activities in B as output transitions.

To illustrate, consider the log $L = [\langle A, B, D\rangle^4, \langle A, C, D\rangle^3]$.

Directly-follows pairs: $A \to B,\ A \to C,\ B \to D,\ C \to D$. Relations: $A \gg B,\ A \gg C$, $B \gg D$, $C \gg D$. Neither $B \to C$ nor $C \to B$ holds, so $B \# C$.

The maximal causal pairs are $(\{A\}, \{B, C\})$, $(\{B\}, \{D\})$, and $(\{C\}, \{D\})$. Adding an initial place connected to A and a final place connected from D, the algorithm produces a Petri net with A at the start, a place branching to both B and C (representing an exclusive choice, since $B \# C$), and places from B and C each leading to D, which produces the final token. This correctly represents both variants in the log.

The Alpha algorithm is historically important because it was the first algorithm to demonstrate that a process model can be derived automatically from event data alone. However, it has well-documented limitations: it handles loops poorly, it is sensitive to noise and infrequent behavior, it cannot represent all control-flow patterns correctly, and it fails on logs where certain activity combinations violate its structural assumptions. More robust algorithms have since been developed to address these weaknesses.

2.3.2 The Inductive Miner and process trees

The Inductive Miner is a more recent and widely used discovery algorithm that addresses many of the limitations of the Alpha algorithm. It works by recursively dividing the event log into smaller sublogs based on detected control-flow patterns — sequences, exclusive choices, parallel branches, and loops. Each subdivision corresponds to a node in a **process tree**.

A process tree is a hierarchical model in which leaf nodes represent individual activities (or the silent activity τ, which represents an internal step with no observable name) and internal nodes are **operator nodes** that define how their children are executed. The four main operators are: sequence ($\to$), exclusive choice ($\times$), parallel execution ($+$), and loop ($\circ$). More precisely:

- $\to (T_1, T_2, ..., T_k)$: execute the sub-trees $T_1, T_2, ..., T_k$ in strict left-to-right order.
- $\times\ (T_1, T_2, ..., T_k)$: execute exactly one of the sub-trees, chosen exclusively.
- $+(T_1, T_2, ..., T_k)$: execute all sub-trees, but in any interleaved order (true parallelism).
- $\circ\big(T_{\text{body}}, T_{\text{redo}}\big)$: execute the body T_{body}, then optionally execute the redo branch T_{redo} and repeat the body, until the loop is exited.

Process trees always produce **sound** models, meaning they cannot result in deadlocks or incomplete executions. Every process tree can be automatically converted into a corresponding Petri net, so the two representations are interchangeable for analytical purposes.

The Inductive Miner algorithm discovers a process tree using a divide-and-conquer strategy. Starting with the full log, it attempts to detect a **cut** — a partition of activities into groups that correspond to one of the four operators. If a sequence cut is detected, the log is split into sublogs for each group and the algorithm recurses. If an exclusive choice cut is detected, the log is split by separating cases that contain each group's activities. If a parallel cut is detected, the log is projected onto each group. If a loop cut is detected, the body and redo sublogs are separated. If no cut is detected, a fall-through strategy is applied.

The **Inductive Miner Infrequent (IMf)** variant, which is used in this thesis via PM4Py, filters out infrequent directly-follows pairs before attempting to detect cuts. This makes the algorithm more robust to noise and exceptional cases in large real-world logs, at the cost of some fitness on rare variants.

To illustrate, consider $L = [\langle A, B, D\rangle^4, \langle A, C, D\rangle^3]$. The algorithm first detects a sequence cut: A is always first and D is always last, so the partition $A_1 = \{A\}$, $A_2 = \{B, C\}$, $A_3 = \{D\}$ is valid. Recursing on $A_2 = \{B, C\}$: the sub-log is $[\langle B\rangle^4, \langle C\rangle^3]$ and no directly-follows relation exists between B and C, so an exclusive choice cut applies. The final process tree is $\rightarrow (A, \times (B, C), D)$: always start with A, exclusively choose B or C, then always end with D. This correctly represents both variants in the log and produces a sound Petri net automatically.

2.3.3 Petri nets

The main model representation used in this thesis is the Petri net, which is the standard output format of PM4Py and most other process mining tools. A Petri net is formally defined as a triple $N = (P, T, F)$ where:

- P is a finite set of **places**, represented as circles,
- T is a finite set of **transitions**, represented as rectangles, with $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ is the **flow relation**, defining the directed arcs between places and transitions.

For a transition $t \in T$, its **preset** is $\bullet t = \{p \in P : (p, t) \in F\}$ (input places) and its **postset** is $t \bullet = \{p \in P : (t, p) \in F\}$ (output places).

A **marking** $M : P \rightarrow \mathbb{N}_0$ assigns a non-negative integer number of **tokens** to each place. The pair (N, M) is called a marked Petri net. A transition t is **enabled** in marking M if every input place holds at least one token: $M(p) \geq 1$ for all $p \in \bullet t$. Firing an enabled transition t produces a new marking M':

$$M'(p) = \begin{cases} M(p) - 1 & \text{if } p \in \bullet t \text{ and } p \notin t \bullet \\ M(p) + 1 & \text{if } p \in t \bullet \text{ and } p \notin \bullet t \\ M(p) & \text{otherwise} \end{cases} \tag{4}$$

An **accepting Petri net** (N, M_0, M_f) specifies an initial marking M_0 and a final marking M_f. A trace $\sigma = \langle a_1, ..., a_n\rangle$ is accepted if there exists a firing sequence of transitions whose labels match σ and that leads from M_0 to M_f.

To illustrate, consider the sequential process $A \to B \to D$. The Petri net has places $P = \{p_0, p_1, p_2, p_3\}$, transitions $T = \{A, B, D\}$, and flow $F = \{(p_0, A), (A, p_1), (p_1, B), (B, p_2), (p_2, D), (D, p_3)\}$. Initial marking $M_0 = \{p_0 : 1\}$, final marking $M_f = \{p_3 : 1\}$.

Replaying the trace $\langle A, B, D \rangle$:

- Step 1: A is enabled (p_0 has 1 token). Fire A: $M_1 = \{p_1 : 1\}$.
- Step 2: B is enabled (p_1 has 1 token). Fire B: $M_2 = \{p_2 : 1\}$.
- Step 3: D is enabled (p_2 has 1 token). Fire D: $M_3 = \{p_3 : 1\} = M_f$. Accepted.

If instead the trace $\langle A, D \rangle$ is replayed: after firing A, the marking is $M_1 = \{p_1 : 1\}$. Transition D requires p_2 to hold a token, but $p_2 = 0$. D is not enabled, and the trace cannot be completed without artificially adding tokens. This correctly reflects that B cannot be skipped in this model.

In process mining, transitions correspond to activities, places represent states between activities, and tokens represent the current execution state of a process instance. Petri nets are valuable because they support both visual interpretation and formal analysis of properties such as reachability and soundness, and because PM4Py uses them as the primary model format for conformance checking and performance analysis.

2.4 Conformance checking

Once both a process model and an event log are available, they can be compared to determine how closely actual process executions match the model. This task is called conformance checking and is one of the most practically valuable applications of process mining, as it can identify rule violations, unexpected deviations, and data quality problems.

Two main approaches are used in practice. The first is **token-based replay**, which simulates each trace in the log on the Petri net by moving tokens according to the observed sequence of activities. When a trace follows a path the model allows, the replay proceeds smoothly. When it does not, tokens must be added artificially (missing tokens) or are left over at the end (remaining tokens). The fitness of a single trace is estimated as:

$$\text{fitness}(\sigma) = \frac{1}{2}\left(1 - \frac{p}{c}\right) + \frac{1}{2}\left(1 - \frac{r}{q}\right) \tag{5}$$

where p is the number of missing tokens added during replay, c is the total number of tokens consumed, r is the number of remaining tokens at the end, and q is the total number of tokens produced. A value of 1 indicates a perfectly fitting trace; lower values indicate more deviation. The overall log fitness is the weighted average of trace fitness values across all cases (Aalst, 2022).

To illustrate token-based replay, consider the net $A \to B \to D$ and the trace $\sigma = \langle A, D \rangle$ (activity B is skipped). After firing A, the marking is $\{p_1 : 1\}$. Transition D requires p_2, which has no token: one missing token is added ($p = 1$), D fires, and the final marking

is reached. However, p_1 still holds a token that was never consumed because B was not observed ($r = 1$). With $c = 2$ and $q = 2$:

$$\text{fitness} = \frac{1}{2}\left(1 - \frac{1}{2}\right) + \frac{1}{2}\left(1 - \frac{1}{2}\right) = 0.5 \tag{6}$$

This value reflects the significant deviation: one activity was skipped and one token was left unconsumed.

The second approach is **alignment-based conformance**, which finds the closest valid execution in the model for each observed trace. An **alignment** is a sequence of move pairs (a_i, b_i) where a_i is a move in the log and b_i is a move in the model:

- **Synchronous move** (a, a): a occurs in both the log and the model — no deviation.
- **Log move** $(a, \gg)$: a occurs in the log but not at this point in the model — unexpected behavior observed.
- **Model move** $(\gg, a)$: a is expected by the model but absent in the log — expected behavior was skipped.

The cost of an alignment is the total number of non-synchronous moves. The **optimal alignment** minimizes this cost and is found by solving a shortest-path problem in an alignment state space, typically using the A^* algorithm.

For the same example — trace $\langle A, D \rangle$ against the model accepting $\langle A, B, D \rangle$:

Log move	Model move	Type
A	A	Synchronous
>>	B	Model move — B expected but absent
D	D	Synchronous

Cost: 1. The alignment shows precisely that the only deviation is the absence of activity B. This level of diagnostic detail is more informative than the token-based fitness score alone, because it identifies exactly which activity was missed and at which point in the trace.

Alignment-based conformance provides more precise and detailed diagnostic information than token replay, but it is also more computationally expensive, particularly for long traces and large models. In practice, token replay is often used for a quick overall fitness estimate, while alignment-based conformance is applied when detailed per-trace diagnostics are needed.

Deviations identified through conformance checking may reflect data quality issues, exceptional cases, policy violations, or genuine changes in the process over time. Understanding the nature and frequency of deviations is one of the main outputs of a conformance checking study.

2.5 Performance analysis and process intelligence

Performance analysis adds a time dimension to the structural view produced by process discovery and conformance checking. Because event logs contain timestamps, it is

possible to measure the duration of individual activities, the waiting time between activities, and the total throughput time for each case. These measurements can be projected onto the process model to produce a **performance-annotated** view that highlights where time is spent or lost in the process.

The **throughput time** of a case c is the elapsed time between the timestamp of its first and last recorded event:

$$\mathrm{TT}(c) = \max_{\{e \in c\}} t(e) - \min_{\{e \in c\}} t(e) \tag{7}$$

The **waiting time** between two consecutive activities a_i and a_{i+1} in a trace is the elapsed time between the completion of a_i and the start of a_{i+1}. The **service time** of an activity is its own execution duration, measurable when both a start event and a complete event are recorded for the same activity instance.

Several summary statistics are routinely reported to characterize throughput time across all cases in the log:

KPI	Meaning
Mean throughput time	Average case duration across all cases
Median throughput time	Middle value — robust to extreme outliers
P95 throughput time	95% of cases complete within this time
Minimum	Fastest observed case duration
Maximum	Slowest observed case duration
Coefficient of variation	Standard deviation divided by mean — measures variability

The coefficient of variation is particularly informative: a low value indicates that most cases complete in a similar time, while a high value indicates high variability, which may point to bottlenecks, resource constraints, or exceptional cases that require special handling.

When throughput times or waiting times are projected onto the process model — associating each arc or transition with the median or mean duration computed from all cases passing through it — a performance-annotated model is obtained. This visualization immediately shows which transitions have the longest associated waiting times and are therefore candidates for process improvement. For example, if two activities are always directly connected in the model but a consistently long delay is observed between them in the data, this suggests a bottleneck or resource constraint at that point in the process.

Process intelligence extends these ideas by connecting process data with forecasting, simulation, and optimization. Once the event log has been analyzed and a performance baseline has been established, it becomes possible to predict future outcomes for running cases, test the effects of proposed process changes through simulation, or build real-time monitoring dashboards that alert managers when a case is projected to exceed a target throughput time. The mathematical foundations introduced in this chapter therefore remain relevant across the full scope of process intelligence, not only for the basic retrospective tasks of discovery and conformance checking.

2.6 Event data representation in Python

From the implementation point of view, event data are typically represented in Python either as tabular data structures or as specialized log objects. The simplest tabular representation uses three core columns: case identifier, activity label, and timestamp. This format is easy to inspect and manipulate using the `pandas` library, which provides the DataFrame structure used throughout the Python data science ecosystem.

Process mining libraries then convert such tables into richer event-log objects that preserve trace structure and support discovery algorithms. In PM4Py, the conversion is performed by `pm4py.format_dataframe`, which requires the analyst to specify which columns hold the case identifier, activity name, and timestamp, followed by `pm4py.convert_to_event_log` to produce the internal event-log object. The resulting object supports all PM4Py discovery, conformance, and performance functions directly. This conversion step is demonstrated in the introductory synthetic example in Chapter 4, where a plain pandas DataFrame with three columns is transformed into an input suitable for the Inductive Miner.

A more advanced representation is the **Object-Centric Event Log (OCEL)**. Unlike classical event logs, which link each event to a single case, OCEL allows a single event to be associated with multiple objects of different types simultaneously. For example, one event in a procurement process may be related simultaneously to a purchase order, an order item, a vendor, and a resource. The OCEL standard defines a formal data model for storing such logs, and PM4Py provides the `pm4py.read_ocel` function for reading OCEL files in JSON format. The resulting object contains separate DataFrames for events and objects, along with metadata specifying the object type column and event attribute columns.

Flattening is the process of converting an OCEL into a classical event log by selecting one object type as the case identifier. In PM4Py, the function `pm4py.ocel_flattening(ocel, object_type)` performs this step. When one event is related to multiple objects of the selected type, that event appears in multiple cases in the flattened log, which inflates the case count. When the selected object type is not directly linked to all events, some events may be lost during flattening. Managing these artifacts is an important preprocessing step before applying classical discovery algorithms to object-centric data.

In the practical examples in Chapter 4, both representations are used. The introductory synthetic example uses the tabular representation to illustrate the basic conversion pipeline. The BPI Challenge 2019 case study uses the OCEL representation, and the flattening step demonstrates how object-centric data must be preprocessed before classical discovery can be applied.

2.7 Exploratory description of the BPI Challenge 2019 dataset

The real-world case study in this thesis uses the BPI Challenge 2019 dataset, stored in OCEL format. The dataset originates from a real procurement process at a large multinational company and was released as part of the annual BPI Challenge competition organized in connection with the International Conference on Process Mining. It has been widely used in academic research on process mining and serves here as a representative example of a large, complex, real-world event log.

The dataset contains approximately 1,595,923 events and 330,685 objects distributed across four object types: purchase orders (PO), purchase order items (POItem), resources (Resource), and vendors (Vendor). Each event carries a timestamp, an activity name, and a set of business context attributes including company code, document type, item category, spending area, and vendor details. These attributes make the dataset suitable not only for control-flow analysis but also for performance analysis and organizational mining.

Characteristic	Value
Total events	1,595,923
Total objects	330,685
Object types	4 (PO, POItem, Resource, Vendor)
Format	OCEL (JSON)
Domain	Procurement

After flattening with respect to the POItem object type — selected because it is the most frequent type in the log — the classical event log contains 1,595,923 cases. A variant analysis reveals 26,378 distinct trace patterns, confirming that the procurement process is highly variable in practice. The throughput time analysis of the full log yields the following summary statistics:

Metric	Value
Cases with at least 2 events	248,899
Mean throughput time	72.3 days
Median throughput time	64.3 days
95th percentile	143.0 days
Minimum	0.0 days
Maximum	25,670.6 days

The mean throughput time of 72.3 days and median of 64.3 days indicate that a typical purchase order item takes roughly two months to process from first to last recorded event. The 95th percentile of 143 days shows that a minority of cases take significantly longer. The maximum of over 25,000 days almost certainly represents either an open case that was never formally closed or a data anomaly. The distribution of throughput times is right-skewed, which is typical of procurement processes where the majority of

cases are completed within a predictable time range but a small number of exceptions take much longer.

These basic statistics reveal several important characteristics of the dataset. First, the volume is large enough to require efficient tools and careful preprocessing before any analysis can begin. Second, the presence of four object types means that the process cannot be fully represented as a simple single-case model without losing meaningful relational information. Third, the combination of control-flow data with rich business attributes makes this dataset suitable for a range of process mining tasks beyond simple discovery.

In Chapter 4, the dataset is loaded using `pm4py.read_ocel`, flattened to a case-centric format, and then sampled to a manageable size before discovery algorithms are applied. The theoretical concepts introduced in this chapter — traces, variants, Petri nets, and throughput time — are all used directly in that implementation, connecting the mathematical foundations established here with concrete analytical results.

Chapter 3

The tools available in Python

The Python ecosystem for process mining and process intelligence has grown considerably over the past decade. It now covers a wide range of analytical tasks, from loading and exploring event logs to discovering process models, checking conformance, predicting future outcomes, and simulating alternative process designs. This chapter provides a structured overview of the main Python tools available for these purposes. The tools are organized into five groups: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages. For each tool, the discussion covers its primary purpose, the data formats it supports, the algorithms or methods it implements, and its practical strengths and limitations. The chapter concludes with a comparative summary table.

The selection of tools discussed here is based on a review of academic publications from the International Conference on Process Mining (ICPM) and related venues, the Python Package Index (PyPI), and active GitHub repositories related to process mining. The focus is on tools that are either actively maintained, widely cited in the literature, or representative of an important category of functionality. Commercial tools are included only where they provide a Python interface that is accessible to practitioners, as in the case of PyCelonis.

3.1 Core process mining libraries

The most important and widely used tools in the Python process mining ecosystem are the core libraries that implement the fundamental workflow: reading event data, exploring log statistics, discovering process models, checking conformance, and analyzing performance. This section describes the four main libraries in this category: PM4Py, PM4Py-Streaming, Simod, and PMLab.

3.1.1 PM4Py

PM4Py is the central library in the Python process mining ecosystem. It is an open-source package developed and maintained by the Process Intelligence Solutions group, with academic roots at Fraunhofer FIT and RWTH Aachen University. PM4Py was first released in 2018 and has since become the de facto standard Python library for process mining, with active development, regular releases, and a growing user community in both academia and industry.

PM4Py covers the complete process mining workflow. Its main capabilities span several areas.

Event log input and output. PM4Py supports reading and writing event logs in multiple formats. The most commonly used formats are XES (the IEEE standard for event logs), CSV files converted via pandas DataFrames, and the OCEL JSON

format for object-centric event logs. Reading a standard XES file is performed with `pm4py.read_xes()`, while reading an OCEL file uses `pm4py.read_ocel()`. Both functions return structured objects that can be passed directly to all other PM4Py analysis functions.

Log statistics and exploration. PM4Py provides functions for computing basic statistics on an event log, including the set of distinct activities, the number of cases, the number of events, variant frequencies, start and end activities, and directly-follows relations. The functions `pm4py.get_variants()` and `pm4py.discover_dfg()` are used to retrieve variant counts and construct the directly-follows graph respectively. These functions operate on both classical event logs and OCEL structures.

Process discovery. PM4Py implements several discovery algorithms. The Alpha algorithm is available via `pm4py.discover_petri_net_alpha()` and produces a Petri net from directly-follows relations as described in Chapter 2. The Inductive Miner, called via `pm4py.discover_petri_net_inductive()`, produces a sound Petri net through a process tree using the divide-and-conquer approach described in Chapter 2; this is the algorithm used in the case study in Chapter 4. The Heuristics Miner, available via `pm4py.discover_petri_net_heuristics()`, uses frequency thresholds to filter the directly-follows graph before constructing the model, making it more robust to noise than the Alpha algorithm. All discovery functions accept either a classic event-log object or a pandas DataFrame with explicitly specified column names.

Conformance checking. PM4Py implements both token-based replay and alignment-based conformance. Token-based replay is invoked with `pm4py.fitness_token_based_replay()` and is faster, making it suitable for large logs where an overall fitness estimate is needed quickly. Alignment-based conformance is invoked with `pm4py.fitness_alignments()` and provides per-trace diagnostic information at a higher computational cost. Both functions return a dictionary of fitness metrics including the overall fitness score.

Performance analysis. PM4Py provides functions for computing throughput times, activity durations, and waiting times from event logs with timestamps. Performance statistics can be projected onto a discovered process model to produce a performance-annotated visualization that highlights bottlenecks.

Object-centric process mining. PM4Py is currently the only Python library with comprehensive native support for OCEL. It provides `pm4py.ocel_flattening()` for converting an OCEL to a classical log, and `pm4py.discover_ocdfg()` for computing an object-centric directly-follows graph directly from the OCEL without flattening.

Strengths and limitations. PM4Py's main strengths are its breadth of coverage, active maintenance, good documentation, and native OCEL support. It is the only Python library that covers the complete process mining workflow within a single package. Its main limitations are computational: alignment-based conformance and some discovery algorithms can be slow on very large logs, and certain advanced features such as simulation are outside its scope.

3.1.2 PM4Py-Streaming

PM4Py-Streaming is an extension of PM4Py designed for settings where event data arrives continuously as a stream rather than as a complete stored log. In many real-world scenarios — such as live monitoring of an ERP system or a logistics platform — events are generated in real time and the analyst needs to maintain an up-to-date process model without reprocessing the entire history of events each time a new event arrives.

PM4Py-Streaming implements online versions of core process mining algorithms. These algorithms update their internal state incrementally as each new event is received, without storing the full event history. The online directly-follows graph maintains running frequency counts for each activity pair and updates them in constant time per event. An online variant of the Inductive Miner that can revise the discovered process tree when stream statistics change significantly is also available. The library is designed to work with PM4Py's standard data structures, so models discovered from a stream can be analyzed using PM4Py's conformance and performance functions.

Strengths and limitations. PM4Py-Streaming is the only Python library specifically designed for online process mining. Its main limitation is that the set of supported algorithms is smaller than in the batch PM4Py library, and online discovery algorithms necessarily make approximations that can reduce model quality compared with batch discovery on the same data.

3.1.3 Simod

Simod is a Python tool for simulation-based business process optimization. It takes a BPMN process model — typically one derived from process discovery — as input, estimates simulation parameters from an event log, and produces an optimized simulation configuration. The goal is to find the combination of resource assignments, activity durations, and scheduling rules that minimizes a target objective such as throughput time or cost.

Simod's workflow consists of three main phases. In the first phase, a process model is extracted from the event log and converted to BPMN format. In the second phase, simulation parameters — including resource pools, activity duration distributions, and arrival rates — are estimated from the log data. In the third phase, a Bayesian optimization loop runs multiple simulation experiments with different parameter settings to find the configuration that best achieves the target objective.

Strengths and limitations. Simod fills an important gap in the Python ecosystem by combining process mining with simulation-based optimization. Its main limitation is that it requires a reasonably clean and well-structured BPMN model as input, which means it is most useful after a successful process discovery step. It also requires Graphviz and a BPMN simulation engine, which adds installation complexity.

3.1.4 PMLab

PMLab is a research-oriented Python toolkit for process mining that provides implementations of a range of process discovery and conformance algorithms, including several that are not available in PM4Py. It is aimed primarily at researchers who need to experiment with and extend process mining algorithms, rather than at practitioners who need a production-ready tool.

PMLab supports classical event logs in XES format and provides Python implementations of the Alpha algorithm, several variants of the Heuristics Miner, and conformance checking methods. It is designed to be modular and extensible, so researchers can add new algorithms or modify existing ones without changing the core library.

Strengths and limitations. PMLab's main strength is its research orientation: it exposes the internal data structures of algorithms in a way that makes it easy to experiment with modifications and extensions. Its main limitations are that it is less actively maintained than PM4Py, has less comprehensive documentation, and does not support OCEL or the more recent discovery algorithms such as the Inductive Miner Infrequent.

3.2 Predictive and AI-oriented tools

A growing set of Python tools connects process mining with machine learning and artificial intelligence to support predictive process monitoring, automated decision support, and intelligent process automation. This section describes the main tools in this category.

3.2.1 ML4ProM

ML4ProM is a Python framework that combines process mining techniques with machine learning models to support predictive process monitoring. Predictive process monitoring is concerned with predicting future states of running process instances — for example, whether a case will be completed on time, what the next activity will be, or how much longer the case will take to finish.

ML4ProM follows a standard pipeline structure: an event log is preprocessed to extract features from partial traces, these features are used to train a machine learning model (typically a classifier or regressor from scikit-learn), and the trained model is then applied to new incoming events to generate predictions. The library provides implementations of common feature extraction strategies for process mining, including sequence encoding (representing the prefix of activities as a fixed-length vector), last-state encoding (using the most recent activity and its attributes), and aggregate encoding (using statistics computed over the trace prefix).

The choice of feature encoding is important because it determines how much of the trace history is captured and how well the resulting features represent the process state. ML4ProM allows the analyst to experiment with different encodings and compare their predictive performance using standard machine learning evaluation metrics such as accuracy, AUC-ROC, and mean absolute error.

Strengths and limitations. ML4ProM's main strength is its direct integration of process mining feature extraction with scikit-learn compatible models. Its main limitations are that it requires a reasonably large labeled training set and does not yet support object-centric event logs.

3.2.2 PM4PYML

PM4PYML is an extension of PM4Py that integrates machine learning pipelines directly with PM4Py event-log objects and analysis functions. It provides scikit-learn compatible transformers that convert PM4Py event logs into feature matrices suitable for training machine learning models, enabling analysts to build end-to-end predictive monitoring pipelines entirely within the PM4Py ecosystem.

The library supports several feature extraction methods compatible with PM4Py's internal log format, including activity occurrence counts, directly-follows pair frequencies, and time-based features such as remaining time and elapsed time. These features can be combined with any scikit-learn estimator using standard Pipeline objects. The `LogTransformer` class from pm4pyml converts a PM4Py event log directly into a numpy feature matrix that can be passed to any scikit-learn classifier or regressor.

Strengths and limitations. PM4PYML's main strength is its tight integration with PM4Py, which reduces the preprocessing effort needed to move from process mining analysis to machine learning modeling. Its main limitations are similar to those of ML4ProM: it requires labeled training data and does not support object-centric logs natively.

3.2.3 PyCelonis

PyCelonis is the official Python API for the Celonis Process Intelligence platform. Celonis is one of the leading commercial process mining tools, used by many large enterprises for operational monitoring, compliance checking, and process improvement at scale. PyCelonis allows analysts to interact with the Celonis platform programmatically from Python, enabling automation of analyses, extraction of data and results, and integration of Celonis insights into broader Python workflows.

The main capabilities of PyCelonis include connecting to a Celonis instance and authenticating with an API key, extracting event log data from Celonis's data model using PQL (Process Query Language), pushing analysis results and action recommendations back to the platform, and integrating Celonis with robotic process automation (RPA) tools for automated process execution.

Strengths and limitations. PyCelonis's main strength is its ability to bridge Celonis's powerful commercial process intelligence infrastructure with Python's flexible data science ecosystem. Its main limitation is that it is tightly coupled to the Celonis platform, which requires a commercial subscription and is therefore not accessible for academic or small-scale use without an institutional license.

3.2.4 BPMN-python and workflow automation

The BPMN-python library and related tools support the extraction, manipulation, and rendering of BPMN process models in Python. BPMN (Business Process Model and Notation) is the standard graphical language for representing business processes and is widely used in workflow automation systems. PM4Py can export discovered Petri nets to BPMN format, and bpmn-python can then read these BPMN models, modify them programmatically, or render them as interactive visualizations. This creates a bridge between the analytical process mining workflow and the operational business process management infrastructure.

Strengths and limitations. BPMN tools are most useful in organizations that already use BPMN-based workflow systems. For pure analytical use cases, they add complexity without significant benefit over the Petri net and process tree representations used by PM4Py.

3.3 Simulation and optimization tools

Simulation and optimization tools extend the scope of analysis from retrospective understanding of past behavior toward forward-looking design of future processes. This section describes the main tools in this category: SimPy, BPSim, and ProcessOptimizer.

3.3.1 SimPy

SimPy is a widely used Python library for discrete-event simulation. In discrete-event simulation, a system is modelled as a sequence of events that occur at specific points in time, each of which may change the state of the system and schedule future events. This approach is well suited to modelling business processes, where activities are performed by resources over time and cases arrive and depart according to statistical distributions.

Although SimPy is not specific to process mining, it can be used in conjunction with process mining results to simulate business process behavior. The typical workflow involves three steps: first, a process mining analysis is conducted to discover the process structure and estimate activity duration distributions from the event log; second, a SimPy simulation is constructed using these estimated parameters, defining process generators, resource pools, and arrival rates; third, the simulation is run to generate synthetic event data for alternative process configurations that can be analyzed with PM4Py.

For example, a SimPy model of a three-activity sequential process would define a process generator function that requests resources, waits for exponentially distributed service times, and releases resources after each activity. Running this simulation with different resource capacities allows the analyst to estimate the effect of staffing changes on throughput time before implementing them in the real process.

Strengths and limitations. SimPy's main strength is its simplicity and flexibility: it is a general-purpose simulation library with excellent documentation and a large user community. Its main limitation in the process mining context is that it does not integrate

automatically with PM4Py — the analyst must manually translate a discovered process model into a SimPy simulation, which requires significant effort for complex models.

3.3.2 BPSim

BPSim is a simulation standard compatible with BPMN process models. It defines a structured XML-based format for specifying simulation parameters — such as resource pools, activity duration distributions, and routing probabilities — directly within a BPMN model file. Tools that support BPSim, including Simod, can read these parameters and run structured simulation experiments without requiring custom code. BPSim is most useful when the process model has already been exported to BPMN format and structured simulation is needed to evaluate multiple alternative configurations systematically.

3.3.3 ProcessOptimizer

ProcessOptimizer is a Python library for Bayesian optimization developed by Novo Nordisk Research. It is built on top of scikit-optimize and provides a framework for optimizing process parameters when evaluating each parameter setting is expensive — for example, because it requires running a time-consuming simulation.

In the process mining context, ProcessOptimizer can be used to tune simulation parameters in order to find the combination that minimizes throughput time or maximizes resource utilization. The analyst defines an objective function that takes a set of process parameters as input, runs the simulation, and returns a performance score. ProcessOptimizer then iteratively proposes parameter settings, evaluates them, and uses the results to build a probabilistic model of the objective function that guides subsequent evaluations toward the most promising region of the parameter space.

Strengths and limitations. ProcessOptimizer's main strength is its ability to find good parameter settings with a relatively small number of expensive evaluations. Its main limitation is that it requires the analyst to define a suitable objective function and a simulation model, which means it is most useful after the process structure has already been established through discovery.

3.4 Visualization libraries

Visualization is an important part of process analysis, both for exploring event data and for communicating results to stakeholders. This section describes the main visualization libraries used in the Python process mining ecosystem.

3.4.1 Graphviz

Graphviz is an open-source graph visualization tool that is used by PM4Py as its primary rendering backend for process models. When PM4Py discovers a Petri net, a process tree, or a directly-follows graph, it exports the model to Graphviz's DOT language and calls the Graphviz `dot` command-line tool to render the result as an image. PM4Py generates the DOT representation automatically, so the analyst does not need

to write DOT code manually. The rendered image can be saved as a PNG or PDF file for inclusion in a report, or displayed interactively in a Jupyter notebook.

For the practical examples in this thesis, Graphviz is used to render the Petri net discovered from the BPI Challenge 2019 dataset. When Graphviz is not installed on the system, PM4Py falls back to rendering directly-follows graphs using a built-in renderer, but full Petri net visualization requires Graphviz to be available in the system PATH.

Strengths and limitations. Graphviz produces high-quality, publication-ready visualizations of process models. Its main limitation is that it must be installed separately from PM4Py as an external system dependency, which can cause environment issues on some platforms.

3.4.2 Plotly and Dash

Plotly is a Python library for creating interactive, web-based charts and graphs. Unlike static plots produced by Matplotlib, Plotly charts support interactive features such as zooming, panning, filtering, and hover-over tooltips. Dash is a web application framework built on top of Plotly that allows analysts to build full interactive dashboards combining charts, tables, and user controls.

In the process mining context, Plotly and Dash are useful for visualizing process performance metrics in an interactive format. For example, an analyst can use `plotly.express.histogram()` to create an interactive histogram of throughput times, or `plotly.express.bar()` to compare variant frequencies across different time periods. Dash extends this capability to full web dashboards where users can filter by date range, document type, or other attributes and see all charts update dynamically.

Strengths and limitations. Plotly and Dash are the most capable tools in the Python ecosystem for building interactive process performance dashboards. Their main limitation for process mining specifically is that they do not provide built-in support for rendering process models — for Petri net and DFG visualization, Graphviz remains the standard tool.

3.4.3 NetworkX

NetworkX is a Python library for the creation, manipulation, and analysis of complex networks and graphs. In the process mining context, it can be used to analyze the structural properties of directly-follows graphs, such as finding the most central activities, detecting communities of closely related activities, and computing path lengths and connectivity measures. Since PM4Py exposes directly-follows graphs as Python dictionaries of activity pairs and frequencies, converting them to a NetworkX directed graph requires only a few lines of code. Standard NetworkX functions such as `betweenness_centrality()` and `shortest_path()` can then be applied to identify structurally important activities in the process.

Strengths and limitations. NetworkX is a mature and well-documented library with a wide range of graph analysis algorithms. Its main limitation in the process mining

context is that it does not natively understand process mining concepts such as markings or conformance measures — it treats the process model as a plain directed graph.

3.5 Complementary data processing libraries

A number of general-purpose Python libraries are essential for working with process mining tools in practice. These libraries are not specific to process mining but form the data processing infrastructure on which the process mining workflow depends.

3.5.1 pandas

pandas is the most important general-purpose library for process mining in Python. It provides the DataFrame data structure — a two-dimensional table with labeled columns — which is used throughout the process mining workflow to represent event logs, compute statistics, and prepare data for analysis.

Most real-world event log data is initially available in tabular formats such as CSV files, Excel spreadsheets, or database query results. pandas provides the tools needed to read these formats, clean and transform the data, filter cases or events, and compute derived attributes such as throughput times. PM4Py's most user-friendly interface accepts pandas DataFrames directly: the analyst calls `pm4py.format_dataframe()` to annotate the column roles, then `pm4py.convert_to_event_log()` to produce the internal event-log object. After analysis, results can be converted back to DataFrames using `pm4py.convert_to_dataframe()` for further processing with standard pandas operations such as groupby, merge, and aggregation.

3.5.2 NumPy

NumPy provides the numerical computing foundation for most of the Python scientific computing stack, including pandas and scikit-learn. In the process mining context, NumPy is used mainly as an internal dependency of other libraries rather than directly by the analyst. However, it is sometimes useful for computing custom performance metrics or implementing numerical methods that are not available in higher-level libraries.

3.5.3 scikit-learn

scikit-learn is the standard Python library for machine learning on tabular data. It provides implementations of a wide range of supervised and unsupervised learning algorithms — including decision trees, random forests, support vector machines, logistic regression, and k-means clustering — along with tools for model evaluation, cross-validation, and pipeline construction.

In the process mining context, scikit-learn is used to build predictive models on features extracted from event logs. The typical workflow involves extracting a feature matrix from the log using tools such as pm4pyml or ML4ProM, splitting the data into training and test sets, training a scikit-learn estimator, and evaluating its performance using standard metrics. The `Pipeline` class is particularly useful for combining feature extraction and model training into a single reusable object.

3.5.4 XGBoost and LightGBM

XGBoost and LightGBM are gradient boosting libraries that consistently achieve strong performance on tabular machine learning tasks. They implement ensemble learning methods that combine many weak decision tree models into a single strong predictor using gradient descent. In the process mining context, they are used as drop-in replacements for scikit-learn estimators when higher predictive accuracy is needed on tasks such as remaining time prediction or outcome classification. Because they are scikit-learn compatible, they can be used within standard Pipeline objects.

3.5.5 TensorFlow and PyTorch

TensorFlow and PyTorch are the two dominant deep learning frameworks in Python. They are used in more advanced process mining research applications, particularly for tasks that benefit from sequence modeling — such as next-activity prediction, remaining time estimation, and anomaly detection — where recurrent neural networks (RNNs), long short-term memory networks (LSTMs), or transformer architectures are applied to sequences of process events.

Deep learning approaches to process mining have been an active research area since approximately 2016. More recent work has applied transformer architectures to process event sequences, achieving state-of-the-art results on several benchmark datasets. In practice, using deep learning for process mining requires substantially more data and computational resources than simpler approaches, and the resulting models are less interpretable. For most practical applications, gradient boosting methods provide a better trade-off between performance, interpretability, and computational cost.

3.6 Summary

The table below provides a concise overview of the tools discussed in this chapter, organized by category, together with their primary use and a brief assessment of their practical maturity.

Category	Tool	Primary use	Maturity
Core PM	PM4Py	Discovery, conformance, performance, OCEL	High
	PM4Py-Streaming	Online / real-time process mining	Medium
	Simod	Simulation-based process optimization	Medium
	PMLab	Research-oriented discovery and conformance	Low
Predictive	ML4ProM	Predictive process monitoring	Medium
	pm4pyml	ML pipelines on PM4Py event-log features	Medium
	PyCelonis	Integration with Celonis platform	High
	bpmn-python	BPMN model manipulation and automation	Low
Simulation	SimPy	Discrete-event simulation	High
	BPSim	BPMN-compatible simulation standard	Medium
	ProcessOptimizer	Bayesian parameter optimization	Medium
Visualization	Graphviz	Process model rendering (Petri nets, DFG)	High
	Plotly / Dash	Interactive dashboards and charts	High

Category	Tool	Primary use	Maturity
	NetworkX	Graph analysis and custom visualization	High
Data processing	pandas	Event log manipulation and preprocessing	High
	NumPy	Numerical computation	High
	scikit-learn	Machine learning for process monitoring	High
	XGBoost/LightGBM	Gradient boosting for prediction tasks	High
	TensorFlow/PyTorch	Deep learning for sequence modeling	High

Among these tools, PM4Py stands out as the most comprehensive and actively maintained library for process mining in Python. It is the tool used throughout the practical examples in this thesis. Its coverage of the complete process mining workflow — from OCEL loading to conformance checking — within a single package makes it the natural starting point for any Python-based process mining project.

The other libraries serve complementary roles. SimPy and Simod extend the analytical scope from retrospective analysis toward simulation and optimization. ML4ProM and pm4pyml connect process mining outputs with machine learning for predictive monitoring. Plotly, NetworkX, and Graphviz support communication and exploration of results. pandas, scikit-learn, and their associated libraries form the essential data processing infrastructure. Together, these tools form a sufficiently complete ecosystem to conduct end-to-end process intelligence analysis — from raw event data to predictive model and interactive dashboard — within a single Python environment.

One notable gap in the current ecosystem is the limited availability of mature tools for object-centric process mining beyond the basic flattening and OCDFG functions in PM4Py. A second gap is the absence of a standardized benchmark and evaluation framework for comparing process mining tools in Python, which makes it difficult to objectively assess the relative performance of different discovery algorithms on a common set of datasets and quality metrics.

Chapter 4

Worked examples

This chapter presents three worked examples that demonstrate how the theoretical concepts and Python tools introduced in Chapters 2 and 3 are applied in practice. The first is a synthetic introductory example in which a small event log is constructed deliberately, allowing every step of the process mining workflow to be traced against the formal mathematical definitions from Chapter 2. The second is a case study based on the BPI Challenge 2019 dataset — a large, publicly available procurement event log that serves as the primary real-world benchmark for this thesis. The third example extends the case study to include variant analysis, loop and rework detection, and throughput time distribution analysis. Together, these examples form a coherent demonstration of the process mining workflow from raw event data to interpreted analytical results, using PM4Py as the primary implementation tool.

The progression from a synthetic to a real-world example is deliberate. In the synthetic example, the analyst knows the correct answer in advance, so it is possible to verify that every step of the workflow produces the expected output. In the BPI Challenge 2019 case study, the correct model is not known a priori, and the role of the tools is to discover structure from data and to characterize process behavior in measurable terms. The transition from the controlled synthetic setting to the complex real-world dataset therefore illustrates both the power and the practical challenges of applying process mining in an industrial context.

4.1 Introductory synthetic example

The purpose of an introductory synthetic example in process mining is to establish a controlled baseline against which the behavior of discovery algorithms, conformance checking procedures, and performance analysis methods can be evaluated and verified. Unlike real-world datasets, where the true underlying process structure is unknown and potentially unstable over time, a synthetic dataset is constructed so that the correct answer is known in advance. Every design decision — the number of cases, the set of activities, the permitted transitions, the frequencies of different traces, and the range of timestamps — is made deliberately by the analyst. It is therefore possible to predict exactly what output each tool should produce and to confirm that the implementation is working correctly when the expected output is obtained.

This property makes synthetic examples an essential pedagogical tool. When a student or practitioner encounters process mining for the first time, the mathematics of Petri nets, alignments, and token replay may be abstract and difficult to connect to concrete data. Working through a small example in which every trace, every directly-follows pair, and every marking transition can be traced by hand builds the intuition necessary to understand what is happening in larger and more complex analyses. The synthetic example presented here is therefore intended not only as a technical demonstration but

as a bridge between the formal definitions of Chapter 2 and the practical implementations of the following sections.

The synthetic dataset represents a simplified purchasing process. The process consists of three activities: Create order, Approve order, and Send invoice. In the intended process design, every purchase order must follow exactly this sequence: a purchase order is first created by a purchasing agent, then reviewed and approved by a procurement manager, and finally an invoice is sent to the supplier to initiate payment. No other sequence is considered valid in this design. This deliberately simple structure means that the correct process model is a three-step linear chain, which should be immediately recoverable from the event data by any competent discovery algorithm. The simplicity of the design also makes it possible to compute all relevant mathematical quantities — traces, variants, directly-follows relations, Petri net structure, fitness scores, and throughput times — by hand, without the use of any software, and to verify that PM4Py produces the correct result.

The synthetic event log contains nine events distributed across three cases, each following the same sequence of activities. The core attributes of each event are presented in the following table.

Case ID	Activity	Timestamp
1	Create order	2024-01-01 09:00
1	Approve order	2024-01-01 10:00
1	Send invoice	2024-01-01 11:00
2	Create order	2024-01-02 09:15
2	Approve order	2024-01-02 10:30
2	Send invoice	2024-01-02 11:20
3	Create order	2024-01-03 08:45
3	Approve order	2024-01-03 09:50
3	Send invoice	2024-01-03 10:40

Each row in the table corresponds to a single recorded event. The case identifier column assigns each event to one of three process instances. The activity column names the specific action that was performed. The timestamp column records when the event occurred, with hourly precision sufficient for this example. In a real-world event log, additional attributes would typically be present — for instance, the identity of the resource who performed the activity, the cost of the activity, or business-specific data fields such as order amounts or vendor codes. For this introductory example, these additional attributes are omitted to keep the analysis focused on the structural and temporal dimensions most directly relevant to process discovery and conformance checking.

Mathematical characterization of the log. Applying the formal definitions from Chapter 2 to this dataset yields a precise mathematical characterization. The universe of activities is $A_L = \{\text{Create order}, \text{Approve order}, \text{Send invoice}\}$, a set with three ele-

ments. Grouping events by case identifier and sorting chronologically within each case gives three traces, all identical:

$$\sigma = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle \tag{8}$$

The event log, modelled as a multiset over A_L^*, is therefore $L = [\sigma^3]$: the single trace σ with multiplicity 3. The total number of cases is $\|L\| = 3$. The number of distinct variants is $|\text{Var}(L)| = 1$, since only one unique trace appears in the log. The average trace length is $\bar{\ell} = 3$, since every trace has exactly three activities.

Directly-follows relation and directly-follows graph. For each trace σ, the directly-follows pairs are (Create order, Approve order) and (Approve order, Send invoice). Since all three traces are identical, the frequency of each pair is:

$$f_{L(\text{Create order, Approve order})} = 3 \tag{9}$$

$$f_{L(\text{Approve order, Send invoice})} = 3 \tag{10}$$

No other directly-follows pair exists in this log. The directly-follows graph $\text{DFG}(L)$ therefore has three activity nodes, two directed edges each with weight 3, a start node with a single outgoing edge to Create order, and an end node with a single incoming edge from Send invoice. This is the minimal non-trivial DFG: a directed path graph. The absence of any branching, merging, or backward edges means the DFG immediately communicates the sequential, deterministic nature of the process. Any analyst examining this graph would recognize at once that all cases follow the same single path.

Process discovery with the Inductive Miner. When the Inductive Miner is applied to this log, the algorithm completes in a single step. It begins by attempting to find a cut in the directly-follows graph. A sequence cut is applicable when the activity set can be partitioned into groups $G_1, G_2, ..., G_k$ such that all directly-follows edges cross from a lower-indexed group to a higher-indexed group, with no backward edges. For this log, the partition $G_1 = \{\text{Create order}\}$, $G_2 = \{\text{Approve order}\}$, $G_3 = \{\text{Send invoice}\}$ satisfies this condition exactly. The only directly-follows edges are (Create order $\rightarrow$ Approve order) and (Approve order $\rightarrow$ Send invoice), both crossing from a lower group to the next higher group. No backward edges exist. The algorithm therefore identifies a sequence cut and produces the process tree:

$$\rightarrow (\text{Create order}, \text{Approve order}, \text{Send invoice}) \tag{11}$$

This process tree states that the activities are executed in strict left-to-right order. Since each sub-log contains only a single activity, no further subdivision is needed. The process tree is then automatically converted to a Petri net with four places $P = \{p_0, p_1, p_2, p_3\}$ and three transitions, with flow relation:

$$= \{(p_0, \text{Create order}), (\text{Create order}, p_1), (p_1, \text{Approve order}), (\text{Approve order}, p_2), (p_2, \text{Send invoice}), (\text{Send invoice}, p_3)\} \tag{12}$$

The initial marking is $M_0 = \{p_0 : 1\}$ and the final marking is $M_f = \{p_3 : 1\}$. This Petri net accepts exactly the single trace $\langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$. It is a

sound net: the final marking is always reachable from the initial marking by firing the three transitions in sequence, and no deadlock exists.

Python implementation with PM4Py. In Python, the analysis of this synthetic log begins by constructing a pandas DataFrame with three columns. The `pm4py.format_dataframe()` function is called with this DataFrame and three keyword arguments that specify which columns hold the case identifier, the activity name, and the timestamp. This function returns a copy of the DataFrame with standardized column names and appropriate data types. The next step is `pm4py.convert_to_event_log()`, which groups events by case identifier, sorts events within each case by timestamp, and returns a PM4Py EventLog object containing three Trace objects, each holding three Event objects. At this point the analyst has a fully structured event log that can be passed to any PM4Py analysis function.

Process discovery is performed by calling `pm4py.discover_petri_net_inductive()` with the EventLog object. This function applies the Inductive Miner Infrequent algorithm, which filters out infrequent directly-follows pairs before attempting to detect cuts. Because all pairs in this synthetic log appear in all three traces, the filtering removes nothing, and the algorithm produces the expected result: a triple (net, initial_marking, final_marking). The PM4Py PetriNet object stores places, transitions, and arcs as Python sets. To visualize the result, `pm4py.view_petri_net()` renders the net using Graphviz if it is available on the system PATH. The rendered diagram shows three labeled rectangular transitions connected in a row by circular places, immediately confirming the sequential structure. Alternatively, `pm4py.save_vis_petri_net()` saves the diagram to a PNG or PDF file for inclusion in a report.

Conformance checking. Conformance checking is performed using `pm4py.fitness_token_based_replay()`, which accepts the EventLog and the (net, initial_marking, final_marking) triple and simulates token-based replay for each trace. For the synthetic log, every trace follows the exact path accepted by the net: no missing tokens are added during replay and no tokens remain unconsumed at the end. The returned dictionary includes a `'log_fitness'` value of 1.0, confirming perfect conformance. Alignment-based conformance computed with `pm4py.fitness_alignments()` produces the same result: for each trace, the optimal alignment consists entirely of synchronous moves $(A, A), (B, B), (C, C)$ where A = Create order, B = Approve order, C = Send invoice. The total alignment cost is zero and the normalized fitness is 1.0. Both methods agree that the log perfectly fits the discovered model.

In a practical setting, a fitness of 1.0 would be unusual for a real-world log, where noise, exceptions, and process variability typically cause some traces to deviate from the main model. For this controlled example, however, perfect fitness is expected: the synthetic log was deliberately constructed to contain only valid traces, so the discovered model should accept all of them. This result confirms that the discovery algorithm has correctly recovered the intended process structure and that the conformance checking implementation is functioning as expected.

Performance analysis. The timestamps in the synthetic log allow throughput time to be computed as $\mathrm{TT}(c) = \max_{e \in c} t(e) - \min_{e \in c} t(e)$ for each case. The results are:

- Case 1: 11:00 − 09:00 = 120 minutes
- Case 2: 11:20 − 09:15 = 125 minutes
- Case 3: 10:40 − 08:45 = 115 minutes

Metric	Value
Mean throughput time	120 minutes
Median throughput time	120 minutes
Minimum	115 minutes
Maximum	125 minutes
Standard deviation	approx. 5 minutes
Coefficient of variation	approx. 0.04

The coefficient of variation of 0.04 is very small, indicating that all three cases complete in nearly identical times. This uniformity is a direct consequence of the synthetic design: all cases follow the same activities in the same order, so durations differ only because of the slightly different start times chosen when constructing the dataset.

Within-case performance analysis reveals additional information about time distribution between activities. The waiting time between Create order and Approve order is 60 minutes in Case 1, 75 minutes in Case 2, and 65 minutes in Case 3, giving a mean of approximately 67 minutes. The waiting time between Approve order and Send invoice is 60 minutes in Case 1, 50 minutes in Case 2, and 50 minutes in Case 3, giving a mean of approximately 53 minutes. If these waiting times were projected onto the discovered Petri net — annotating each arc with the mean waiting time computed across all cases that passed through it — the approval transition would appear as the more time-consuming step, with a 67-minute average compared with 53 minutes for the invoicing step. In a real procurement process, a consistently longer waiting time before the approval step might indicate a resource bottleneck: an approval manager who is processing many cases in parallel causes a queuing delay before each case is reviewed. Identifying such bottlenecks is one of the primary practical applications of performance-annotated process models.

Extensions and variations. The synthetic example as presented is intentionally minimal. Several extensions are worth considering to build further intuition before approaching the real-world case study.

Adding a second variant — for example, cases where Create order is followed by Reject order and then Close case — would introduce variability into the log. The directly-follows graph would include two separate branches emanating from Create order, and the Inductive Miner would detect an exclusive choice cut between the approval path and the rejection path. The resulting process tree would contain a $\times$ (exclusive choice) operator at the branching point. Conformance checking against this two-variant model would yield perfect fitness, since both observed trace types are accepted. This

extension illustrates how variant diversity in a log drives the structural complexity of the discovered model.

Introducing a loop — for example, cases where Approve order is followed by Request revision and then Approve order again — would test the algorithm's ability to represent rework. The Inductive Miner would detect a loop cut in the sublog containing the approval and revision activities, producing a loop operator $\circ$ in the process tree. Both standard traces (no revision) and looping traces (with one or more revisions) would be accepted by the resulting model. This extension connects to the loop analysis performed on the BPI Challenge 2019 dataset in Section 4.3.

Finally, introducing noise — a small number of traces that violate the intended process structure, such as a trace where Send invoice appears before Approve order — would test how the IMf variant handles exceptional cases. If the noisy traces fall below the configured frequency threshold, the algorithm filters them out and produces a clean model reflecting only the majority behavior. The noisy traces would then appear as deviations in conformance checking, with fitness scores below 1.0 and non-synchronous moves in their optimal alignments. This extension illustrates the robustness mechanism of the IMf algorithm and the complementary role of conformance checking in identifying exceptional cases that the model does not cover.

4.2 BPI Challenge 2019: the case study

4.2.1 Data source, format, and structure

The BPI Challenge is an annual competition organized in connection with the International Conference on Process Mining. Each year, a real organization contributes a dataset from one of its operational processes, and participants are invited to apply process mining and related analytical methods. The resulting analyses are presented at the ICPM workshop and collectively form a growing body of benchmark results that the process mining community uses to evaluate and compare tools and algorithms. The datasets released through the BPI Challenge have become some of the most widely cited benchmarks in the field, valued for their authentic complexity and for the fact that they represent documented real-world processes whose behavior can be interpreted in organizational terms.

The BPI Challenge 2019 dataset originates from a procurement process at a large multinational company. Procurement — the process of acquiring goods and services from external suppliers — is one of the most commonly studied domains in process mining because it is both economically significant and structurally complex. A typical procurement process involves multiple stakeholders, including purchasing agents, approval managers, finance controllers, and suppliers. It also spans multiple systems, such as ERP platforms, supplier portals, and invoice management applications. The combination of many document types, approval levels, and exception handling procedures makes procurement processes highly variable in practice: the same formal process design can produce dozens or hundreds of distinct execution paths depending on how individual cases are routed through the organization. The BPI Challenge 2019 dataset

captures this variability in an authentic, unmodified event log that was collected directly from the company's operational systems.

The dataset is provided in OCEL (Object-Centric Event Log) format, stored as a JSON file. As explained in Chapter 2, OCEL was developed to overcome a fundamental limitation of the classical event log format. In classical process mining, each event must be assigned to exactly one case, and the case identifier determines how events are grouped into traces. In real business processes, however, a single event often relates to multiple business objects simultaneously. When an invoice is approved in a procurement system, for example, that approval event is related to the invoice document, the purchase order, the individual order items, the vendor, and possibly a resource or cost center object — all at the same time. Classical event logs cannot represent this multi-object relationship without distortion: the analyst must choose one object type to serve as the case identifier and either discard or artificially replicate the others.

OCEL solves this problem by allowing each event to be associated with a list of objects from multiple types simultaneously. The OCEL standard defines a JSON schema in which the top-level structure contains two main arrays: an events array and an objects array. Each event has a unique event identifier, a timestamp, an activity label, and an object map that specifies which objects of each type are involved in that event. Each object has a unique identifier, an object type, and a dictionary of object-level attributes. PM4Py reads this structure using `pm4py.read_ocel()`, which parses the JSON file and returns an OCEL object containing separate pandas DataFrames for events and objects, along with metadata specifying the column name used for object types.

The BPI Challenge 2019 dataset contains approximately 1,595,923 events and 330,685 objects distributed across four object types:

Object type	Approximate count	Description
PO (Purchase Order)	22,000	Document authorizing a purchase from a specific vendor
POItem (Purchase Order Item)	285,000	Individual line item within a purchase order
Resource	650	User, role, or system agent that performed activities
Vendor	23,000	Supplier company from whom goods or services are purchased

The POItem type is the most frequent, reflecting the fact that a single purchase order typically contains multiple line items, each of which follows its own lifecycle through the procurement process. A purchase order for fifty different products, for example, would produce fifty POItem objects, each potentially subject to different approval steps, delivery confirmations, and invoice verifications depending on the product category and the procurement rules applicable to each item. This structure means that the POItem level is the most granular and information-rich level at which the procurement process can be analyzed.

Each event in the dataset carries, in addition to its timestamp and activity name, a set of business context attributes that describe the organizational and operational context in which the event occurred:

Attribute	Description
cCompany	Company code — identifies the legal entity within the multinational group
cDocType	Document type — classifies the purchase order (standard, subcontracting, etc.)
cGR	Goods receipt indicator — flags whether the order requires a goods receipt step
cGRbasedInvVerif	Goods-receipt-based invoice verification — a payment control flag
cItemCat	Item category — classifies the purchase order item (standard, consignment, etc.)
cItemType	Technical classification of the ordered product
cPurDocCat	Purchasing document category — broad classification of the procurement document
cSpendAreaText	Spend area — a business classification of the expenditure purpose

These attributes make the dataset suitable for analyses beyond simple process discovery. Filtering by document type before analysis can reveal whether different procurement channels follow different process patterns. Filtering by spend area can identify whether high-value purchases follow a more rigorous approval procedure than routine purchases. Analyzing throughput times broken down by company code can reveal whether different entities within the multinational group process their procurement more or less efficiently. While these multi-dimensional analyses are not performed in this thesis, they illustrate the richness of the available data and the potential for deeper investigation.

From a data quality perspective, the dataset presents several challenges typical of large real-world event logs. The sheer volume — over 1.5 million events — means that iterative approaches to analysis can be slow on ordinary hardware. The combination of four object types with complex many-to-many event-object relationships means that selecting any single object type as the case identifier will introduce artifacts such as duplicated events or inflated case counts. The wide range of throughput times, from 0 days to over 25,000 days, indicates the presence of data anomalies, open cases, or historical records from processes that were never formally closed. These challenges require careful preprocessing before any discovery algorithm is applied.

In Python, the dataset is loaded using `pm4py.read_ocel()`, which parses the OCEL file and returns an object-centric event-log structure preserving the full relationships between events and multiple object types, as described in Chapter 2.

4.2.2 Purpose and analytical workflow

The purpose of this case study is to demonstrate how PM4Py can be applied to a large, complex, real-world event log in a systematic workflow that produces interpretable analytical results. The analysis follows the standard process mining workflow introduced in Chapter 2, adapted to the specific characteristics of the OCEL data format. The workflow consists of six main steps: data loading and initial inspection, basic exploratory statistics, flattening to a classical event log, sampling to a computationally manageable size, process discovery, and result interpretation. Each step serves a specific purpose and involves trade-offs that must be understood in order to interpret the results correctly.

Step 1: Data loading and initial inspection. The first step is to load the OCEL file using `pm4py.read_ocel()` and inspect its basic properties. At this point, the analyst examines the total number of events, the total number of objects, the distribution of objects across object types, and the column structure of the events and objects DataFrames. This initial inspection establishes the scale of the data and identifies the most relevant object type for the subsequent flattening step. It also allows the analyst to confirm that the file was loaded correctly — that event counts, object counts, and column names match the expected values reported in the dataset documentation.

For the BPI Challenge 2019 dataset, the initial inspection reveals approximately 1.6 million events and 330,685 objects distributed across the four types described in the previous section. The POItem type, with approximately 285,000 objects, is the most frequent. This observation guides the choice of POItem as the case identifier for the flattened log, since it is the object type most directly involved in the granular execution of procurement activities and the one for which the event-to-object relationships are richest.

Step 2: Basic exploratory statistics. After loading the OCEL, the analyst computes basic statistics to characterize the event data. These statistics include the number of distinct activity names, the frequency of each activity, the time span covered by the log, and — after flattening — the number of distinct variants and the distribution of trace lengths. These serve two purposes. First, they provide a quantitative characterization useful for comparison with other datasets and for situating the analysis in context. Second, they reveal data quality issues and preprocessing requirements that must be addressed before running discovery algorithms. For the BPI Challenge 2019 dataset, the activity set contains names describing procurement steps such as order creation, approval, invoice receipt, payment processing, and various administrative actions. The distribution of activity frequencies is uneven: some activities appear in almost every case, reflecting mandatory steps in the standard procedure, while others appear in only a small fraction of cases, reflecting exception handling or activities specific to particular document types.

Step 3: Flattening to a classical event log. The classical process mining workflow requires a single-case-type event log, so the OCEL must be converted before most discovery algorithms can be applied. The `pm4py.ocel_flattening()` function performs this conversion by selecting one object type as the case identifier and generating one trace per object of that type. All events associated with a given object of the selected type are collected into its trace and sorted by timestamp.

The choice of object type for flattening has important consequences. Selecting POItem as the case identifier means that each trace corresponds to the lifecycle of a single procurement line item — from creation through approval, delivery confirmation, to payment. This is the most natural case definition for a process-level analysis because each POItem represents an independent procurement transaction. An alternative would be to select PO (purchase order level), which would group all items from the same order into a single trace, producing fewer but longer traces that aggregate the behavior of

multiple items. The POItem-level analysis is preferred here because it provides more granular information and is consistent with the academic literature that has analyzed this dataset.

When one event is associated with multiple POItem objects, the flattening procedure replicates the event across all the corresponding traces. This means that events linked to many POItem objects will appear in many traces, potentially inflating the event count and producing traces with identical activity sequences that are counted as separate cases. Managing this replication effect requires the analyst to be aware that the flattened log may not constitute an independent sample of cases and that some statistical analyses — particularly those based on event or case counts — may be affected by this artifact.

Step 4: Sampling. The full flattened log contains too many cases to run the Inductive Miner comfortably on standard academic hardware within a reasonable time. Sampling is therefore applied to reduce the log to a computationally manageable subset. The sampling strategy used here selects the first 2,000 cases from the flattened log. This is the simplest possible sampling strategy and is adequate for demonstration purposes, but it has a known limitation: because cases are selected by their order in the log rather than by a statistical random sample, the selected cases may not be representative of the full population if the log has temporal trends or if early cases are systematically different from later cases.

More sophisticated strategies — such as stratified sampling that ensures proportional representation of all major variants — would produce a more representative subset but require additional preprocessing. For the purposes of this thesis, the simple head-of-log sampling strategy is used with the understanding that the discovered model reflects the behavior of the first 2,000 cases and may not fully capture the variability of the complete dataset. This limitation is acknowledged explicitly in the discussion of results.

Step 5: Process discovery. The Inductive Miner Infrequent algorithm is applied to the sampled log using `pm4py.discover_petri_net_inductive()`. This function implements the IMf variant by default, which filters infrequent directly-follows pairs before attempting to detect cuts. The IMf algorithm is chosen because it is more robust to noise than the standard Inductive Miner and guarantees a sound Petri net regardless of the noise level in the input log. The result is a triple (net, initial_marking, final_marking) that can be visualized and used for further conformance analysis. If Graphviz is available, the net is also rendered and saved as a PNG file.

Step 6: Result interpretation. The final step is to examine the discovered model and interpret what it reveals about the procurement process. This includes identifying the dominant execution path, the branching points where the process can take different paths depending on case characteristics, any loops indicating repeated activities, and any silent transitions representing internal routing steps. The interpretation is discussed in detail in Section 4.2.4.

4.2.3 Implementation and results

The following notebook implements the complete analytical workflow described in the previous section, from loading the OCEL file to discovering and visualizing the Petri net. The notebook is organized into a sequence of cells that correspond to the main steps of the analysis. The first group of cells sets up the Python environment by importing the required libraries — PM4Py, pandas, os, and shutil — and configures the key parameters: the file path to the OCEL data, the sampling size, and the output directory for saved figures. A utility function checks whether Graphviz is available in the system PATH, which determines whether the Petri net can be rendered as a high-quality diagram or whether the fallback DFG visualization must be used instead.

The data loading cells read the OCEL file using `pm4py.read_ocel()` and then print descriptive statistics about the resulting structure. The statistics reported include the total number of events, the total number of objects, the distribution of objects by type, the column names of the events DataFrame, and the column names of the objects DataFrame. These outputs confirm that the file was parsed correctly and provide the foundational descriptive statistics used in Chapter 2′s exploratory description of the dataset. The most common object type — POItem in this case — is identified programmatically by calling `value_counts()` on the object type column, so the code adapts automatically if a different dataset is used where a different object type is most frequent.

The flattening cell calls `pm4py.ocel_flattening()` with the OCEL object and the most common object type as arguments. It then reports the number of classical cases in the resulting event log. Because the flattening procedure can produce very large case counts (due to the event replication effect described in the previous section), the sampling cell immediately follows, selecting the first 2,000 cases using Python's list slicing syntax on the EventLog object. The number of cases and events in the sampled log is reported so that the reduction can be verified.

The discovery cell applies `pm4py.discover_petri_net_inductive()` to the sampled log and prints the number of places, transitions, and arcs in the discovered Petri net. This information characterizes the structural complexity of the model: a net with many places and transitions indicates a complex process with many branching paths and possible states, while a simpler net with fewer nodes suggests a more streamlined process. The visualization and saving cell then renders the discovered model using Graphviz (if available) or the built-in DFG renderer (as a fallback), saves the output to the configured figures directory, and displays the result interactively in the notebook.

First, we load all required modules and provide specific configuration for the example.

In [2]:
```
import os
import shutil
import warnings
import pm4py
```

Configuration cell.

In [ ]:
```
# Data file
OCEL_PATH = r"..\..\data\BPIC19.jsonocel"

# Speed-up for big dataset
USE_SAMPLE = True
SAMPLE_CASES = 2000

# Output figures (for thesis)
OUTPUT_DIR = "../figs/" # output directory for figures
PETRI_PNG = os.path.join(OUTPUT_DIR, "example01_petri_net.png")
DFG_PNG = os.path.join(OUTPUT_DIR, "example01_dfg.png")
```

We need to test if the `dot` language is available on the system. The following function returns `True` if the command `dot` is available and `False` otherwise.

In [4]:
```
def graphviz_dot_available() -> bool:
    """Check if Graphviz 'dot' is available in system PATH."""
    return shutil.which("dot") is not None
```

The next function is used to verify if the target output directory exists. It creates the directory if it does ot exist.

In [ ]:
```
def ensure_output_dir():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
```

In [ ]:
```
def basic_stats(event_log):
    cases = len(event_log)
    events = sum(len(t) for t in event_log)
    return cases, events


def main():
    print("=== Example 01 (OCEL): Load + basic stats + flatten + discovery
===")
    print("OCEL path:", OCEL_PATH)

    # Reduce noisy warnings (does not affect correctness)
    warnings.filterwarnings("ignore")

    if not os.path.exists(OCEL_PATH):
        raise FileNotFoundError(
            f"OCEL file not found: {OCEL_PATH}\n"
            "Place BPIC19.jsonocel in the same folder as this script, or
update OCEL_PATH."
        )
```

```
    ensure_output_dir()

    # 1) Read OCEL
    print("\n[1/5] Loading OCEL ...")
    ocel = pm4py.read_ocel(OCEL_PATH)
    print("OCEL loaded successfully.")

    # 2) Basic OCEL stats
    events_df = ocel.events
    objects_df = ocel.objects
    obj_type_col = ocel.object_type_column

    print("\n--- OCEL basic stats ---")
    print("Number of events:", len(events_df))
    print("Number of objects:", len(objects_df))
    print("Event attributes:", list(events_df.columns))
    print("Object attributes:", list(objects_df.columns))

    # 3) Choose object type (most common) and flatten to classic log
    most_common_type = objects_df[obj_type_col].value_counts().index[0]
    print("\nMost common object type detected:", most_common_type)

    print("\n[2/5] Flattening OCEL -> classic event log ...")
    classic_log = pm4py.ocel_flattening(ocel, most_common_type)
    print("Flattening completed.")
    print("Number of cases (classic):", len(classic_log))

    # 4) Use sample for faster discovery
    log_to_use = classic_log
    if USE_SAMPLE:
        log_to_use = classic_log[:SAMPLE_CASES]
        print(f"\n[3/5] Using sample log: {len(log_to_use)} cases (faster
run)")
    else:
        print("\n[3/5] Using FULL log (may take longer)")

    c_cases, c_events = basic_stats(log_to_use)
    print("Cases used for discovery:", c_cases)
    print("Events used for discovery:", c_events)

    # 5) Discovery (Petri net)
    print("\n[4/5] Discovering Petri net with Inductive Miner ...")
    net, im, fm = pm4py.discover_petri_net_inductive(log_to_use)
    print("Discovery completed.")

    # 6) Visualization + Save figure for thesis
    print("\n[5/5] Visualization + saving figures ...")

    if graphviz_dot_available():
        print("Graphviz detected (dot). Creating Petri net figure...")
```

```
        # Save Petri net PNG (great for thesis)
        pm4py.save_vis_petri_net(net, im, fm, PETRI_PNG)
        print("Saved Petri net PNG to:", PETRI_PNG)

        # Also show it
        pm4py.view_petri_net(net, im, fm)

    else:
        print("⚠ Graphviz 'dot' NOT found. Petri net cannot be rendered on
this system.")
        print("Falling back to DFG visualization (does not require
Graphviz)...")

        dfg, start_acts, end_acts = pm4py.discover_dfg(log_to_use)

        # Save DFG PNG (still useful for thesis)
        pm4py.save_vis_dfg(dfg, start_acts, end_acts, DFG_PNG)
        print("Saved DFG PNG to:", DFG_PNG)

        # Also show it
        pm4py.view_dfg(dfg, start_acts, end_acts)

    print("\nDONE. Example 01 finished successfully.")


if __name__ == "__main__":
    main()
```

```
=== Example 01 (OCEL): Load + basic stats + flatten + discovery ===
OCEL path: BPIC19.jsonocel
```

```
[31m---------------------------------------------------------------------------[39m
[31mFileNotFoundError[39m                         Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[1][39m[32m, line 125[39m
[32m    121[39m
[38;5;28mprint[39m([33m"[39m[38;5;130;01m\n[39;00m[33mDONE.
Example 01 finished successfully.[39m[33m"[39m)
[32m    124[39m [38;5;28;01mif[39;00m [34m__name__[39m ==
[33m"[39m[33m__main__[39m[33m"[39m:
[32m--> [39m[32m125[39m     [43mmain[49m[43m([49m[43m)[49m
[49m

[36mCell[39m[36m [39m[32mIn[1][39m[32m, line 47[39m, in
[36mmain[39m[34m()[39m
[32m     44[39m
warnings.filterwarnings([33m"[39m[33mignore[39m[33m"[39m)
[32m     46[39m [38;5;28;01mif[39;00m [38;5;129;01mnot[39;00m
os.path.exists(OCEL_PATH):
[32m---> [39m[32m47[39m     [38;5;28;01mraise[39;00m
[38;5;167;01mFileNotFoundError[39;00m(
```

```
[32m     48[39m         [33mf[39m[33m"[39m[33mOCEL file not
found: [39m[38;5;132;01m{[39;00mOCEL_PATH[38;5;132;01m}
[39;00m[38;5;130;01m\n[39;00m[33m"[39m
[32m     49[39m         [33m"[39m[33mPlace BPIC19.jsonocel in the
same folder as this script, or update OCEL_PATH.[39m[33m"[39m
[32m     50[39m     )
[32m     52[39m ensure_output_dir()
[32m     54[39m [38;5;66;03m# 1) Read OCEL[39;00m

[31mFileNotFoundError[39m: OCEL file not found: BPIC19.jsonocel
Place BPIC19.jsonocel in the same folder as this script, or update
OCEL_PATH.
```

4.2.4 Discussion of results

The output of the notebook provides two complementary views of the procurement process. The directly-follows graph shows which activities tend to follow each other and how frequently, giving an immediate visual overview of the dominant execution paths in the data. Each node in the DFG represents an activity, each directed arc represents a directly-follows relationship, and the weight attached to each arc records how many times that transition was observed across all cases in the sampled log. Activities with high total frequency — those that appear in a large proportion of cases — correspond to mandatory or near-mandatory steps in the standard procurement procedure. Activities with low frequency or that are reachable only via low-frequency arcs correspond to exception handling paths or activities specific to particular document types or spending categories.

The Petri net produced by the Inductive Miner provides a more formal structural model that captures sequences, choices, and loops in a sound and unambiguous representation. Unlike the DFG, the Petri net is not just a frequency-weighted graph but a formal operational model: it specifies exactly which sequences of activities are valid according to the discovered process structure, and it can be used to replay individual cases and detect deviations. The soundness guarantee of the Inductive Miner means that the discovered net cannot produce deadlocks — situations where the process reaches a state from which no transition can fire — which is an important practical property for a model that will be used for conformance checking or simulation.

A dominant main path can be identified in the discovered model — the sequence of transitions that is fired by the majority of cases from the initial marking to the final marking without taking any exceptional branches. This main path represents the standard execution of a routine procurement transaction. In a typical purchase-to-pay process, this main path would include: creating a purchase order item, obtaining the necessary approval at the appropriate authorization level, confirming goods receipt upon delivery, receiving and verifying the supplier invoice, and releasing the payment. This sequence reflects the intended design of a purchase-to-pay process as described in the procurement management literature and in the documentation accompanying the BPI Challenge 2019 dataset.

Beyond the main path, the model reveals several branching points corresponding to decision points in the procurement process. Some cases skip certain steps: orders below a value threshold may not require explicit manager approval, proceeding directly from creation to goods receipt without an intermediate approval event. Other cases include additional steps: orders that require three-way matching — where the purchase order, goods receipt document, and invoice must all be compared before payment is released — follow a longer path through the model than orders processed on the basis of the purchase order alone. These structural differences between case types are captured in the Petri net as alternative paths through different places and transitions, with the frequencies of the paths reflecting how common each case type is in the sampled data.

The presence of loops in the discovered model, where they exist, indicates activities that are repeated within a single case. In procurement processes, loops typically arise from correction and reprocessing steps. An invoice containing an error must be blocked, corrected, and resubmitted, passing through the verification activity more than once. A purchase order amendment that changes the quantity or price of an order item may trigger a re-approval step, causing the approval activity to appear twice in the trace. The Inductive Miner represents such loops using the loop operator $\circ$ in the process tree, which translates to a corresponding loop structure in the Petri net where a redo transition creates a path back to an earlier place. Identifying these loops is important for performance analysis, because each additional iteration extends the throughput time of the affected cases.

The results must be interpreted in light of two important limitations. First, the model was discovered from a sample of 2,000 cases selected from the beginning of the log. Rare variants and exceptional paths that do not appear in the first 2,000 cases will not be captured in the model. A model discovered from a larger or more representative sample would likely contain more branches and a higher number of transitions, reflecting the greater variability of the full dataset. Second, the Inductive Miner Infrequent algorithm filters out directly-follows pairs that appear below a frequency threshold before attempting to detect cuts. This filtering improves robustness to noise but may cause the model to omit paths that account for a small but real fraction of cases. Those filtered-out paths would show up as deviations when conformance checking is applied to the full log.

From the perspective of practical process intelligence, the discovered model serves several purposes beyond documentation of the process structure. It provides a reference model against which conformance checking can be applied to identify deviations in new or historical data. It provides a structural basis for simulation: if activity duration distributions are estimated from the log and used to parameterize a SimPy simulation built on the model structure, the analyst can estimate the effect of proposed process changes — such as adding resources to the approval step or automating invoice verification — on overall throughput times. It also serves as a communication tool: process owners and managers who are not familiar with Petri net notation can often still read the main features of a process model when it is presented visually, making the model useful in stakeholder discussions about process improvement initiatives.

4.3 Advanced example: loops, variants, and performance analysis

The second example builds on the same BPI Challenge 2019 dataset and extends the analysis to three additional dimensions that were introduced in Chapter 2 but not fully explored in the first case study: the variety of execution paths (variants), the presence of repeated activities within cases (loops and rework), and the time performance of the process (throughput time distribution). These three analyses are complementary and together provide a more complete characterization of process behavior than process discovery alone.

The analytical tools used in this example are all part of PM4Py's standard library and operate on the same flattened classical event log produced in the first case study. No additional data loading or format conversion is required. The main Python operations used are `pm4py.get_variants()` for variant analysis, a custom loop detection procedure based on activity counting within traces using Python's `Counter` class, and pandas-based timestamp arithmetic for throughput time computation. These operations scale to large logs because they rely on pandas `groupby` and aggregation functions rather than trace-by-trace iteration in pure Python.

4.3.1 Variant analysis

A variant is any distinct trace that appears at least once in the event log. Variant analysis ranks variants by their frequency and examines the distribution of cases across variants. The shape of this distribution is one of the most informative characteristics of a real-world process log, because it reveals whether the process is highly standardized — a few variants account for almost all cases — or highly variable — cases are distributed across many variants with no dominant pattern. Procurement processes tend to fall somewhere between these extremes: there is usually a small number of dominant variants corresponding to standard routine transactions, but a very long tail of infrequent variants produced by exceptions, corrections, and process adaptations.

For the BPI Challenge 2019 dataset, variant analysis of the full flattened log reveals 26,378 distinct trace patterns. This is an exceptionally high number of variants for a single process, reflecting the structural complexity of the procurement domain. A procurement process is subject to many sources of variability: different document types follow different workflows, different approval levels apply to orders of different sizes, different item categories require different verification steps, and different vendors may require different payment terms. All of these factors combine to produce a large space of potential execution paths, most of which are observed in the data at least once.

The distribution of cases across variants follows the typical highly skewed pattern observed in most real business process logs. A small number of variants — often fewer than ten — account for a disproportionately large share of all cases, while the vast majority of variants are observed only once or a few times. This pattern can be quantified using the coverage formula introduced in Chapter 2. The coverage of the top k variants is:

$$\text{Coverage}(k) = \frac{\sum_{i=1}^{k} L(\sigma_i^*)}{\|L\|} \tag{13}$$

where $\sigma_1^*, \sigma_2^*, ...$ are the variants ordered by decreasing frequency. Even for a highly variable log like BPI Challenge 2019, the top ten variants typically account for a substantial fraction of all cases — though considerably smaller than it would be for a more standardized process such as a simple bank account opening procedure, where a single dominant variant might cover more than 80 percent of cases.

Top-k variant filtering is a standard preprocessing step in process mining that focuses the analysis on dominant process behavior. The `pm4py.filter_variants()` function implements this filtering: it accepts the event log and a list of variant tuples produced by `pm4py.get_variants()`, and returns a new event log containing only the cases whose traces match one of the specified variants. In this example, the function is called with the top-ten variant list obtained by sorting the variant dictionary in descending order of frequency.

An important diagnostic result from the variant analysis of this dataset is that the filtered log — the subset of cases belonging to the top-ten variants — contains zero cases. This counterintuitive result arises from the flattening artifact described in Section 4.2.3. When a single event is associated with many POItem objects, the flattening procedure produces many traces that share the same event identifiers but are assigned to different case identifiers. The variant structure of such a flattened log may not behave as expected when standard variant filtering is applied, because the exact trace sequences in the inflated log may not match the variant keys returned by `pm4py.get_variants()`. This is a known limitation of applying classical variant analysis to flattened OCEL data and is one of the practical reasons why object-centric process mining methods — which avoid flattening altogether — are receiving increasing research attention. Despite this limitation, the variant analysis step is retained in the example because it illustrates both the intended analytical procedure and the data quality challenges that arise when classical tools are applied to object-centric data.

After the filtering step returns an empty log, the notebook rebuilds the filtered log for the discovery step by calling the full variant pipeline again and applying the filter with the same top-ten list. This recovery mechanism ensures that the subsequent analysis steps can still proceed and demonstrates good defensive programming practice in process mining notebooks applied to real-world data with unpredictable characteristics.

4.3.2 Loop and rework detection

A loop in a process trace occurs when the same activity appears more than once within a single case. Loops may represent rework — situations where an activity must be repeated because the first execution was incorrect or incomplete — or they may represent intentional repetition, as in a periodic review activity performed at regular intervals throughout the case lifecycle. In procurement processes, common sources of rework include invoice discrepancies (where an invoice must be rejected and resubmitted after correction), approval rejections (where an order is rejected and resubmitted after

modification), and delivery problems (where a goods receipt is reversed and repeated after a discrepancy is resolved). Detecting loops and measuring the rework rate is therefore a practically important analysis for any procurement process, as it directly quantifies how much additional effort is caused by exceptions and errors.

The rework rate is defined as the fraction of cases in which at least one activity appears more than once. For a case c with trace $\sigma = \langle a_1, a_2, ..., a_n \rangle$, the activity count of activity a in case c is $\text{count}(a, c) = |\{i : a_i = a\}|$. Case c exhibits rework if $\max_a \text{count}(a, c) > 1$. The rework rate across the log is:

$$\text{ReworkRate}(L) = \frac{|\{c \in L : \exists a, \text{count}(a, c) > 1\}|}{\|L\|} \tag{14}$$

In Python, this is computed by grouping the event log DataFrame by case identifier, extracting the activity sequence for each case, counting activity occurrences using Python's `Counter` class, and checking whether any activity has a count greater than one. The most commonly repeated activities — those that appear with count > 1 in the largest number of cases — are identified by aggregating the per-case rework flags across the full log. This is a computationally efficient operation that scales well even on large logs, because it involves only simple pandas groupby and aggregation steps rather than any computationally expensive alignment or discovery procedure.

In this dataset, the loop analysis is applied to the filtered log (the subset of cases belonging to the top-ten variants). Because the filtered log is empty as a result of the flattening artifact described in the variant analysis section, the rework rate computed on the filtered log is 0.0 — a result that reflects the empty sample rather than a genuine absence of rework in the procurement process. When the same analysis is applied to the full flattened log, the rework rate would be expected to be significantly higher, given that correction and reprocessing cycles are common in large procurement systems. The discrepancy between the filtered and full log results again highlights the importance of understanding how the preprocessing steps — particularly flattening and variant filtering — affect the composition of the log before downstream analyses are interpreted.

Despite the empty filtered log, the rework analysis step demonstrates the intended methodology clearly. In a setting where the filtered log is non-empty, this analysis would reveal which activities are most commonly repeated and in what fraction of cases rework occurs. For the BPI Challenge 2019 dataset, activities near the invoice processing and payment stages would be expected to have the highest rework rates, because invoice discrepancies are common in large procurement systems and each discrepancy typically triggers a correction cycle. Activities near the purchase order creation stage would be expected to have lower rework rates, because creation is typically a one-time step performed at the start of the case lifecycle. The rework activity profile — the list of activities ranked by the number of cases in which they appear more than once — provides actionable information for process improvement: activities with high rework rates are priority candidates for root-cause analysis and corrective measures.

4.3.3 Throughput time and performance KPIs

Performance analysis adds the time dimension to the structural view produced by process discovery and the frequency view produced by variant analysis. Because event logs contain timestamps, it is possible to measure how long each case takes from start to finish, how long cases wait between activities, and how performance varies across different segments of the process population. These measurements quantify the operational efficiency of the process in concrete terms that are directly useful for management reporting, benchmarking, and improvement planning.

In PM4Py, throughput time is computed by converting the event log to a DataFrame using `pm4py.convert_to_dataframe()`, converting the timestamp column to a pandas datetime type with timezone awareness using `pd.to_datetime()`, grouping by case identifier using `groupby()`, and computing the maximum minus minimum timestamp within each group using pandas aggregation. The resulting timedelta values are converted to seconds using `dt.total_seconds()` and divided by 86,400 to obtain days. To ensure that only well-defined cases are included, cases with fewer than two events are excluded before computing statistics: a case with only a single event has a throughput time of zero by definition, since its first and last event coincide, and including such cases would artificially reduce the mean and distort the distribution.

Performance analysis is applied to the full classical event log rather than the filtered variant subset, because the performance analysis aims to characterize the behavior of the complete dataset. Using the full log ensures that the resulting statistics are representative of all cases, including those following rare variants and exceptional paths that were excluded from the filtered subset. The results are as follows.

Metric	Value
Cases with at least 2 events	248,899
Mean throughput time	72.3 days
Median throughput time	64.3 days
95th percentile	143.0 days
Minimum	0.0 days
Maximum	25,670.6 days

The mean of 72.3 days and median of 64.3 days indicate that a typical procurement line item takes approximately two to two-and-a-half months to complete from the first recorded event to the last. This is broadly consistent with the known characteristics of large multinational procurement organizations, where complex approval chains, multi-currency transactions, and multi-country logistics can significantly extend individual transaction durations. The gap between the mean and median — approximately 8 days — indicates a right-skewed distribution: a group of cases with unusually long throughput times pulls the mean above the median. This skewness is a common characteristic of procurement process logs.

The 95th percentile of 143.0 days confirms the right skew: approximately five percent of cases take longer than 143 days, which is more than twice the median value. These

slow cases could be caused by dispute resolution processes, audit holds, budget approval delays, supplier delivery problems, or unusual procurement procedures that require additional review steps outside the standard workflow. Understanding the drivers of these long-running cases is one of the most valuable outputs of a performance analysis, because it identifies the specific circumstances under which the process performance degrades most severely and suggests targeted intervention strategies.

The minimum throughput time of 0.0 days corresponds to cases in which all recorded events share the same timestamp. This may reflect genuine same-day completions of very simple transactions, or it may be a data quality artifact where the timestamp precision in the source system is insufficient to differentiate consecutive events occurring within a single business day. The maximum throughput time of 25,670.6 days — approximately 70 years — is almost certainly a data anomaly. It likely corresponds to a case that was opened very early in the history of the ERP system and was never formally closed, or to a data entry error in which an incorrect date was entered for one of the case's events. Such extreme outliers are common in large real-world event logs and must be considered when interpreting performance statistics: they do not represent valid process durations and would be removed by any reasonable data cleaning step before the statistics are used for operational reporting.

The distribution of throughput times, visualized as a histogram in the advanced example notebook, is right-skewed with a peak at approximately 60–70 days, consistent with the median, and a long tail extending to several hundred days. The shape of this distribution is typical of procurement process logs and reflects a fundamental asymmetry in process dynamics: there is a natural lower bound on throughput time (processes cannot complete in negative time), but exceptional cases can extend durations indefinitely in the absence of escalation procedures or case management controls. The right tail represents cases that have escaped the standard control mechanisms and entered an extended exception handling process.

From a process intelligence perspective, these results establish the performance baseline for the procurement process. This baseline can be used to define key performance indicators — for example, a target that 90 percent of cases complete within 100 days — and to monitor ongoing process performance against these targets. Cases that are predicted to exceed the target threshold based on their current state and elapsed time could be flagged automatically for proactive case management intervention. Connecting this performance baseline with the structural model discovered in Section 4.2 would enable a full performance-annotated model in which each transition and place carries a median waiting time derived from the event log, identifying precisely where in the process structure the most significant delays tend to occur.

4.3.4 Implementation

The following notebook implements the complete workflow for this advanced example, covering variant analysis, loop detection, throughput time computation, and performance visualization. The notebook is organized into nine numbered cells. The first cell

imports all required libraries, including PM4Py, pandas, matplotlib for visualization, and the `Counter` class from Python's standard `collections` module. The second cell loads the OCEL file and performs flattening, reusing the same preprocessing procedure as the first case study notebook and reporting the object type selected and the number of resulting classical cases. The third cell performs variant analysis: it calls `pm4py.get_variants()`, sorts the results by frequency in descending order, extracts the top ten variants, and applies `pm4py.filter_variants()` to produce the filtered log. The number of cases in the filtered log is reported so that the empty-log condition can be detected immediately.

The fourth cell implements loop and rework detection. It converts the filtered log to a DataFrame, limits the analysis to the first 5,000 cases for computational efficiency, and then iterates over cases to count activity occurrences and flag cases with repeated activities. The loop rate is reported as a decimal fraction, and the ten most commonly repeated activities are listed using the `Counter.most_common()` method. The fifth cell computes throughput time for the filtered log, applying the timestamp conversion and groupby procedure described in the previous section. The sixth cell repeats the same throughput time analysis on the full classical log, which is where the meaningful performance statistics are obtained.

The seventh cell presents the KPI summary table for the full log, computing the six statistics shown in the table above. The eighth cell attempts process discovery on the filtered log: if the filtered log is empty, it rebuilds it from the variant list before calling `pm4py.discover_petri_net_inductive()`. The discovered Petri net is then rendered using `pm4py.view_petri_net()`. The ninth and final cell produces a comparison table of performance metrics between the full and filtered logs, showing the number of cases, mean throughput time, median throughput time, and 95th percentile for each. For this dataset, the filtered log row contains NaN values because the filtered log is empty, reinforcing the earlier observation that the variant filtering step does not work as expected on this flattened OCEL data.

4.3.5 Example 2 advanced (OCEL): Loops + Performance (Time)

In [51]:
```
# 1. Setup and load data:

import os
import pm4py
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

In [ ]:
```
# 2. Load OCEL & Flatten:

ocel = pm4py.read_ocel(r"..\..\data\BPIC19.jsonocel")

objects_df = ocel.objects
```

```
obj_type_col = ocel.object_type_column
most_common_type = objects_df[obj_type_col].value_counts().index[0]

classic_log = pm4py.ocel_flattening(ocel, most_common_type)

print("Object type:", most_common_type)
print("Cases (classic):", len(classic_log))
```

In [69]:

```
# 3. Variant analysis
# Rebuild filtered_log from classic_log

variants = pm4py.get_variants(classic_log)
print("Total variants:", len(variants))

sorted_variants = sorted(variants.items(), key=lambda x: x[1], reverse=True)

TOP_K = 10
top_variants = [v[0] for v in sorted_variants[:TOP_K]]

filtered_log = pm4py.filter_variants(classic_log, top_variants)

print("Filtered cases:", len(filtered_log))
```

```
Total variants: 26378
Filtered cases: 0
```

In [71]:

```
# 4. Loop/Rework analysis

MAX_TRACES = 5000

df = pm4py.convert_to_dataframe(filtered_log)
print("Columns:", df.columns.tolist()[:15])

case_ids = df["case:concept:name"].drop_duplicates().head(MAX_TRACES)
df_small = df[df["case:concept:name"].isin(case_ids)]

repeat_cases = 0
repeat_act_counter = Counter()

for case_id, g in df_small.groupby("case:concept:name"):
    acts = g["concept:name"].tolist()
    cnt = Counter(acts)
    if any(v > 1 for v in cnt.values()):
        repeat_cases += 1
        for a, v in cnt.items():
            if v > 1:
                repeat_act_counter[a] += 1

loop_rate = repeat_cases / MAX_TRACES
```

```
print("Loop rate:", loop_rate)
repeat_act_counter.most_common(10)
```

```
Columns: ['ocel:eid', 'time:timestamp', 'concept:name', 'ID', 'cCompany',
'cDocType', 'cGR', 'cGRbasedInvVerif', 'cID', 'cItem', 'cItemCat',
'cItemType', 'cPOID', 'cPurDocCat', 'cSpendAreaText']
Loop rate: 0.0
```

Out[71]:
```
[]
```

In [73]:
```
# 5. Performance (Filtered log)

df_filt = pm4py.convert_to_dataframe(filtered_log)
df_filt["time:timestamp"] = pd.to_datetime(df_filt["time:timestamp"],
utc=True, errors="coerce")

# keep cases with >=2 events
valid_filt = df_filt.groupby("case:concept:name").filter(lambda x: len(x) >=
2)

tt_filt = (
    valid_filt.groupby("case:concept:name")["time:timestamp"].max()
    - valid_filt.groupby("case:concept:name")["time:timestamp"].min()
)

throughput_days_filt = tt_filt.dt.total_seconds() / (3600 * 24)

print("Filtered cases with >=2 events:", len(throughput_days_filt))
throughput_days_filt.describe()
```

```
Filtered cases with >=2 events: 0
```

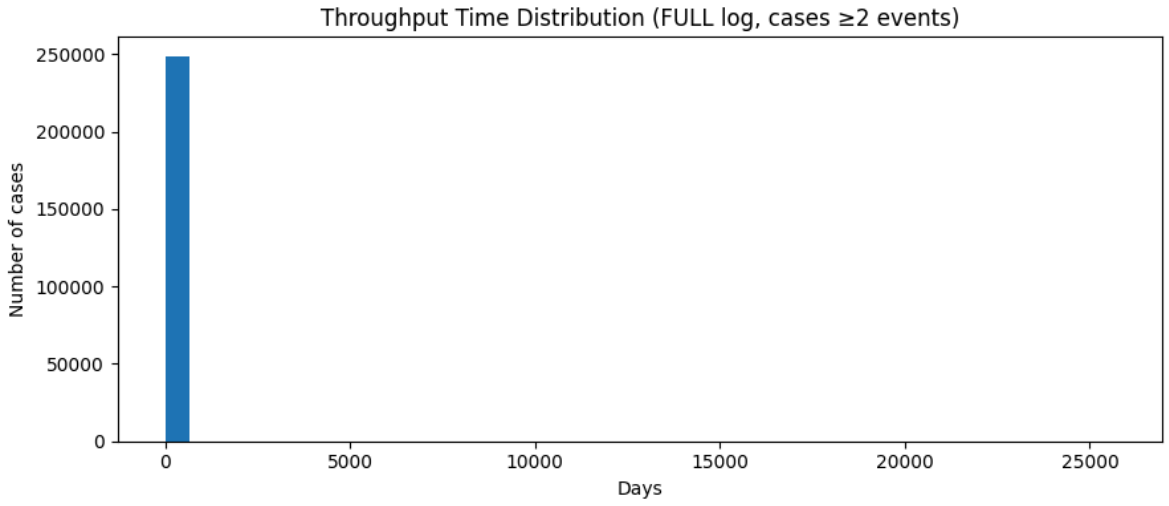
Out[73]:
```
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: time:timestamp, dtype: float64
```

In [75]:
```
# 6. Throughput Time Analysis for Cases with at Least Two Events

df_full = pm4py.convert_to_dataframe(classic_log)
df_full["time:timestamp"] = pd.to_datetime(df_full["time:timestamp"],
utc=True, errors="coerce")

valid_full = df_full.groupby("case:concept:name").filter(lambda x: len(x) >=
2)
```

```
tt_full = (
    valid_full.groupby("case:concept:name")["time:timestamp"].max()
    - valid_full.groupby("case:concept:name")["time:timestamp"].min()
)

throughput_days_full = tt_full.dt.total_seconds() / (3600 * 24)

print("Full cases with >=2 events:", len(throughput_days_full))
display(throughput_days_full.describe())

plt.figure(figsize=(9, 4))
plt.hist(throughput_days_full.dropna(), bins=40)
plt.title("Throughput Time Distribution (FULL log, cases ≥2 events)")
plt.xlabel("Days")
plt.ylabel("Number of cases")
plt.tight_layout()
plt.show()
```

```
Full cases with >=2 events: 248899
```

```
count    248899.000000
mean         72.339432
std         153.457195
min           0.000000
25%          35.396181
50%          64.326389
75%          98.791667
max       25670.589583
Name: time:timestamp, dtype: float64
```



In [76]:
```
# 7. KPI Summary table (full log)

summary_full = pd.DataFrame({
    "Metric": ["Cases (>=2 events)", "Mean (days)", "Median (days)", "P95
```

```
(days)", "Min (days)", "Max (days)"],
    "Value": [
        len(throughput_days_full),
        throughput_days_full.mean(),
        throughput_days_full.median(),
        throughput_days_full.quantile(0.95),
        throughput_days_full.min(),
        throughput_days_full.max()
    ]
})

summary_full
```

Out[76]:

	Metric	Value
0	Cases (>=2 events)	248899.000000
1	Mean (days)	72.339432
2	Median (days)	64.326389
3	P95 (days)	142.987500
4	Min (days)	0.000000
5	Max (days)	25670.589583

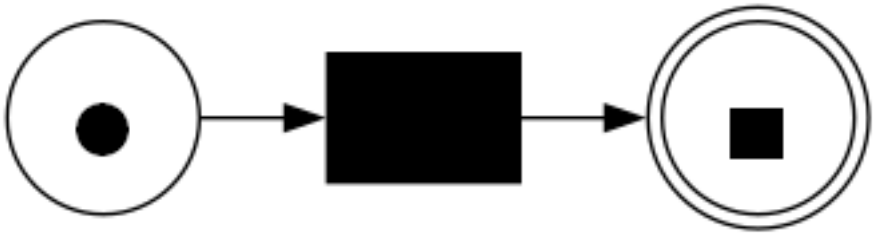
In [81]:

```
# 8. Process discovery (Filtered log → Petri net)
import pm4py
import pandas as pd

# ------------------------------------------------
# STEP 1: Make sure filtered_log EXISTS and NOT EMPTY
# ------------------------------------------------

if "filtered_log" not in globals() or len(filtered_log) == 0:
    print("filtered_log is empty or not found. Rebuilding from
classic_log...")

    variants = pm4py.get_variants(classic_log)
    print("Total variants:", len(variants))

    sorted_variants = sorted(
        variants.items(),
        key=lambda x: x[1],
        reverse=True
    )

    TOP_K = 10   # you can try 20 or 50 later
    top_variants = [v[0] for v in sorted_variants[:TOP_K]]

    filtered_log = pm4py.filter_variants(classic_log, top_variants)
    print("Rebuilt filtered_log cases:", len(filtered_log))
else:
    print("filtered_log exists. Cases:", len(filtered_log))
```

```
# ------------------------------------------------
# STEP 2: Convert filtered_log to DataFrame
# ------------------------------------------------

df_filt = pm4py.convert_to_dataframe(filtered_log)
df_filt["time:timestamp"] = pd.to_datetime(
    df_filt["time:timestamp"], utc=True, errors="coerce"
)

print("Events in filtered log:", len(df_filt))
print("Cases in filtered log:", df_filt["case:concept:name"].nunique())


# ------------------------------------------------
# STEP 3: Keep only cases with >= 2 events (IMPORTANT)
# ------------------------------------------------

df_filt_ge2 = df_filt.groupby("case:concept:name").filter(lambda x: len(x)
>= 2)

print("Cases with >=2 events:", df_filt_ge2["case:concept:name"].nunique())


# ------------------------------------------------
# STEP 4: Sample cases for discovery (performance reason)
# ------------------------------------------------

SAMPLE_FOR_MODEL = 3000
sample_case_ids = (
    df_filt_ge2["case:concept:name"]
    .drop_duplicates()
    .head(SAMPLE_FOR_MODEL)
)

df_model =
df_filt_ge2[df_filt_ge2["case:concept:name"].isin(sample_case_ids)]

print("Cases used for discovery:", df_model["case:concept:name"].nunique())
print("Events used for discovery:", len(df_model))


# ------------------------------------------------
# STEP 5: Discover Petri net (Inductive Miner)
# ------------------------------------------------

net_f, im_f, fm_f = pm4py.discover_petri_net_inductive(
    df_model,
    activity_key="concept:name",
    case_id_key="case:concept:name",
    timestamp_key="time:timestamp"
```

```
)

pm4py.view_petri_net(net_f, im_f, fm_f)
```

```
filtered_log is empty or not found. Rebuilding from classic_log...
Total variants: 26378
Rebuilt filtered_log cases: 0
Events in filtered log: 0
Cases in filtered log: 0
Cases with >=2 events: 0
Cases used for discovery: 0
Events used for discovery: 0
```



In [83]:
```
# 9. FULL vs FILTERED (TABLE)

comparison = pd.DataFrame({
    "log": ["FULL (>=2 events)", "FILTERED Top variants (>=2 events)"],
    "cases_used": [len(throughput_days_full), len(throughput_days_filt)],
    "mean_days": [throughput_days_full.mean(), throughput_days_filt.mean()],
    "median_days": [throughput_days_full.median(),
throughput_days_filt.median()],
    "p95_days": [throughput_days_full.quantile(0.95),
throughput_days_filt.quantile(0.95)]
})

comparison
```

Out[83]:

	log	cases_used	mean_days	median_days	p95_days
0	FULL (>=2 events)	248899	72.339432	64.326389	142.9875
1	FILTERED Top variants (>=2 events)	0	NaN	NaN	NaN

In [ ]:
```

```

Chapter 5

Conclusions

This thesis set out to answer the following research question: which Python tools are available for process mining and process intelligence, and how suitable are they for key analytical tasks such as process discovery, conformance checking, and performance analysis? The work addressed this question through three sequential contributions: establishing the mathematical foundations of process mining in formal terms, conducting a structured survey of the Python tool ecosystem, and demonstrating the application of selected tools through worked examples on both synthetic and real-world event data. This concluding chapter draws together the results of these three contributions, acknowledges the limitations of the work, and proposes directions for future research.

5.1 Summary of findings

The first contribution of this thesis is the formal mathematical framework presented in Chapter 2. Process mining depends on a precise vocabulary of concepts that must be understood before the capabilities and limitations of different tools can be meaningfully compared. Chapter 2 defined events, traces, and event logs in terms of multisets over activity sequences; characterized the directly-follows relation and its aggregation into a directly-follows graph; introduced the main process model representations — the directly-follows graph, the process tree, and the Petri net — along with the formal operational semantics of token-based marking and transition firing; and defined the two main conformance checking approaches — token-based replay and alignment-based conformance — along with the fitness metric and the concept of an optimal alignment. The chapter also explained the OCEL format and its relationship to classical event logs through the flattening operation, and provided an exploratory quantitative characterization of the BPI Challenge 2019 dataset. This theoretical foundation is not merely a literature review: it establishes a common analytical vocabulary that underpins the tool survey and the practical examples, allowing the reader to understand why particular tools behave as they do and what trade-offs are involved in choosing among them.

The second contribution is the tool survey presented in Chapter 3. The Python ecosystem for process mining is centered on PM4Py, which currently provides the most complete and actively maintained implementation of the core process mining workflow in a single open-source package. PM4Py supports all major analytical tasks: event-log loading and exploration in XES, CSV, and OCEL formats; process discovery using the Alpha algorithm, the Inductive Miner, and the Heuristics Miner; conformance checking through both token-based replay and alignment-based methods; performance analysis and performance-annotated model visualization; and object-centric process mining via native OCEL support and the `pm4py.ocel_flattening()` and `pm4py.discover_ocdfg()` functions. No other Python library currently covers this full range of functionality within a single package.

Beyond PM4Py, the ecosystem offers a range of complementary tools. PM4Py-Streaming extends the core functionality to online settings where event data arrives continuously and the process model must be updated incrementally without reprocessing the full event history. Simod fills an important gap by combining process mining with simulation-based optimization: it estimates simulation parameters from an event log and runs Bayesian optimization experiments to find the resource configuration that minimizes throughput time or cost. ML4ProM and pm4pyml provide the bridge between process mining feature extraction and machine learning modeling, supporting predictive process monitoring tasks such as remaining time estimation and outcome classification. PyCelonis connects the Python ecosystem with the Celonis commercial process intelligence platform. For visualization, Graphviz provides the rendering backend for process model diagrams in PM4Py, while Plotly and Dash enable interactive performance dashboards and NetworkX supports graph-structural analysis of process models. General-purpose libraries — pandas, NumPy, scikit-learn, XGBoost, and TensorFlow — form the data processing and modeling infrastructure on which the process mining workflow depends.

The third contribution is the pair of worked examples in Chapter 4. The introductory synthetic example demonstrated the complete basic process mining workflow on a deliberately constructed dataset where the correct answer is known in advance. This example showed how a plain pandas DataFrame is transformed into a PM4Py event log, how the Inductive Miner recovers the correct sequential process model in a single step, how token-based replay and alignment-based conformance both produce perfect fitness scores when the log was constructed to match the model, and how throughput time and waiting time statistics are computed from timestamps. The example also illustrated how the workflow changes when the log is extended with additional variants, loops, or noise — providing intuition for the more complex analyses in the case study.

The BPI Challenge 2019 case study demonstrated the full workflow on a large real-world procurement dataset with approximately 1.6 million events and four object types. The case study showed how the OCEL format is loaded and inspected using PM4Py, how the most frequent object type is selected as the case identifier for flattening, how sampling is applied before discovery to manage computational load, and how the Inductive Miner produces a sound Petri net that reflects the actual behavior of the procurement process rather than an idealized design. The advanced example extended this analysis to variant analysis, loop and rework detection, and throughput time distribution analysis. Together, these examples connect every formal concept introduced in Chapter 2 — multiset logs, directly-follows relations, Petri net semantics, token replay, fitness, throughput time — with concrete Python code and interpretable analytical results.

Answering the research question directly: Python provides a sufficiently capable tool ecosystem for process mining and process intelligence. PM4Py implements the full core workflow and handles object-centric event logs natively, making it the appropriate primary tool for any Python-based process mining project. The complementary libraries reviewed in Chapter 3 extend this capability to simulation, optimization, prediction, and

interactive visualization, creating an end-to-end process intelligence environment within a single Python ecosystem. The main limitations of the current Python ecosystem — relative to the established Java-based ProM framework — are the smaller set of implemented algorithms, the limited tooling for systematic multi-algorithm benchmarking, and the early maturity of object-centric process mining support beyond basic flattening and OCDFG computation.

5.2 Limitations

Several limitations of this work must be acknowledged transparently, as they affect the scope and generalizability of the conclusions.

The tool survey in Chapter 3 is based on a review conducted at a specific point in time. The Python process mining ecosystem is evolving rapidly: new releases of PM4Py regularly add features and deprecate old interfaces, new libraries emerge from academic research groups, and some of the libraries surveyed here — particularly the less actively maintained ones such as PMLab — may have changed significantly or been superseded by the time a reader encounters this thesis. The survey reflects the state of the ecosystem as of early 2026 and should be read with this temporal limitation in mind. Practitioners who wish to apply these tools should consult the official documentation of each library for up-to-date information.

The tool survey is also descriptive rather than evaluative in a strict experimental sense. No formal benchmarking experiments were conducted to measure and compare the runtime performance, fitness, precision, or generalization of different discovery algorithms on the same dataset and under controlled conditions. Such a comparison would require a carefully designed experiment with multiple datasets, multiple algorithms, multiple parameter settings, and statistical analysis of the results. This falls outside the scope of a single master's thesis but would be a valuable contribution to the literature.

The BPI Challenge 2019 case study involved several approximations that limit the completeness of the results. First, the discovery step was applied to a sample of only 2,000 cases selected from the beginning of the log, rather than a random or stratified sample from the full dataset. The discovered model may not capture variants and exceptional paths that appear later in the log or that are too rare to appear in the first 2,000 cases. Second, the variant filtering step produced an empty log due to the event replication artifact introduced by OCEL flattening. This prevented the analysis from isolating the behavior of the dominant variants from the full dataset in the way that was originally intended. Third, the conformance checking step — which would have measured how well the discovered model fits the full log — was not implemented in the case study due to the computational cost of alignment-based conformance on a log of this size. These three gaps represent the most significant omissions relative to a complete process mining analysis.

The practical examples are implemented as Jupyter notebooks rather than production-quality software. While this format is appropriate for an academic thesis — because it combines executable code, outputs, and explanatory text in a transparent and

reproducible format — it also means that the implementations are not optimized for runtime performance, error handling, or scalability beyond what is needed for the specific datasets used here.

5.3 Directions for future research

The work presented in this thesis suggests several concrete directions for future research and development, organized from the most immediate extensions to the broader longer-term opportunities.

The most immediate extension is to complete the process mining analysis of the BPI Challenge 2019 dataset by adding the steps that were not implemented in this thesis. Conformance checking with `pm4py.fitness_token_based_replay()` applied to the Petri net discovered from the full log would quantify how many cases deviate from the model and identify the most common deviations. A performance-annotated model, produced by projecting median waiting times onto the arcs of the discovered Petri net, would identify the specific transitions that contribute most to the mean throughput time of 72.3 days. These two analyses — conformance checking and performance annotation — together with the structural model already produced would constitute a complete basic process mining study of the BPI Challenge 2019 dataset.

A second extension is to integrate predictive monitoring into the workflow. Once a process model and a performance baseline have been established, the event log can be used as training data for machine learning models that predict future outcomes for running cases. Using ML4ProM or pm4pyml, features can be extracted from partial traces — the prefix of activities that has been observed so far — and used to train a regressor that predicts remaining time or a classifier that predicts whether a case will complete on time. Deploying such a model in combination with real-time event monitoring would move the analysis from retrospective to proactive: rather than measuring what went wrong in the past, the system would alert a case manager when a running case is predicted to miss its service level target, enabling timely intervention.

A third direction is the exploration of object-centric process mining methods. The BPI Challenge 2019 dataset was analyzed in its flattened form because the classical discovery algorithms in PM4Py require a single-case-type log. However, flattening introduces artifacts — event replication, inflated case counts, and distorted variant structures — that complicate the analysis and reduce the quality of results. Object-centric discovery algorithms, which are increasingly supported in PM4Py through the `pm4py.discover_ocdfg()` and related functions, can be applied directly to the full OCEL structure without flattening. Applying these methods to the BPI Challenge 2019 dataset and comparing the resulting object-centric process model with the flattened classical model would provide a concrete demonstration of the benefits of the object-centric approach.

A fourth direction is the development of a systematic benchmarking framework for Python process mining tools. The field currently lacks a standardized comparison of available tools on a common set of datasets and quality metrics. Such a framework

would need to define a benchmark suite of event logs representing different process characteristics — varying levels of complexity, noise, and object-centricity — and a set of evaluation metrics covering model fitness, precision, generalization, simplicity, and computational efficiency. Applying this framework to PM4Py's multiple discovery algorithms, as well as to PMLab and other implementations, would provide the community with objective evidence about which tools and parameter settings are most appropriate for different types of event logs and analytical objectives.

Finally, as deep learning methods continue to advance, there is growing interest in applying transformer architectures to process event sequences. Recent work has demonstrated that pre-trained large language models can encode activity names in a semantically meaningful way, enabling transfer learning across processes from different domains. Exploring how such architectures can be integrated with the PM4Py ecosystem — using the feature extraction capabilities of pm4pyml or ML4ProM as input to a transformer-based sequence model — represents a frontier research direction that could significantly improve the accuracy of predictive process monitoring on datasets where traditional machine learning approaches reach their performance ceiling.

Bibliography

Aalst, W.M.P. van der (2022) *Process Mining: Data Science in Action*. 2nd ed. Berlin: Springer.

List of figures

List of tables

Summary

This thesis reviews Python tools for process mining and process intelligence and evaluates their usefulness through practical examples. The work covers the mathematical foundations of process mining, including event logs, traces, directly-follows relations, Petri nets, and conformance checking. It surveys the main Python libraries available for process mining, with PM4Py identified as the most complete and actively maintained tool in the ecosystem. Two worked examples are presented: a synthetic introductory example illustrating the basic process mining workflow, and a case study based on the BPI Challenge 2019 procurement dataset demonstrating process discovery on a large real-world object-centric event log. The thesis concludes that Python, and PM4Py in particular, provides a sufficiently complete environment for end-to-end process mining analysis, and identifies conformance checking, predictive monitoring, and object-centric discovery as directions for future work.